

Flutter Banking Capstone

10 Project Problem Statements

Features, non-functional requirements, API contracts and architecture — aligned to the 60-hour Flutter track

Track: Flutter cross-platform development · Modules 1–7

Projects: PayLite · CardVault · LoanEase · SaveSmart · SpendWise · BillBuddy · BranchQ · OpenAccount · InvestLite · FraudShield

Each includes: Problem statement · personas · prioritised features · acceptance criteria · screens · data model · API contract · NFRs · architecture diagram · edge cases · stretch goals

Reference build: BankEase (Modules 1–6) is the worked example every project follows

Author: Nawaz Bagwan · Illuminate Skills

Contents

How to use this document.....	3
Project selection matrix	4
What every project must demonstrate	6
Baseline functional requirements (all projects)	7
Baseline non-functional requirements	8
Reference architecture	9
Milestones, deliverables and evaluation	11
P01 · PayLite.....	12
P02 · CardVault	18
P03 · LoanEase	23
P04 · SaveSmart	28
P05 · SpendWise	33
P06 · BillBuddy	38
P07 · BranchQ	43
P08 · OpenAccount	48
P09 · InvestLite	53
P10 · FraudShield	58
Appendix · Writing acceptance tests from these criteria	63

Word asks to update fields when the file opens: choose Yes to fill this table.

How to use this document

Each project is a realistic banking product brief, written the way a product owner would hand it to a mobile team. Every project is sized for the capstone (Module 7) and deliberately exercises every module of the track: the same concepts learners practised on BankEase, applied to a new domain without step-by-step code.

Reader	How to use it
Trainer	Assign one project per team of 2–3 (or let teams choose from the selection matrix). Use the rubric at the end of Part A for evaluation. Each project names its "hard part" so reviews can focus there.
Participants	Read Part A first: it contains the requirements that apply to every project. Then read your project in full before writing any code. Build in the order of the milestones.
Scope	Must features are required to pass. Should features separate good from excellent. Could features and stretch goals are for teams that finish early.

Note: Every project reuses the BankEase architecture: feature folders, Riverpod, repositories, a sealed error type, Dio with interceptors, secure storage and biometrics. Nothing here requires a library the course did not teach, apart from those named in a project (for example a charting or camera package).

Project selection matrix

#	Project	Theme	Level	The hard part
P01	PayLite — UPI-style person-to-person payments with QR scan-and-pay	Payments	Advanced	Payment status is not binary
P02	CardVault — Debit and credit card control centre	Cards	Intermediate	Optimistic UI done right
P03	LoanEase — Personal loan application and EMI tracker	Lending	Advanced	A long, interruptible workflow
P04	SaveSmart — Goal-based savings, recurring and fixed deposits	Savings	Intermediate	Financial maths that must be exactly right, plus motion that motivates without getting in the way
P05	SpendWise — Expense analytics and monthly budgets	Personal finance	Intermediate	Fast aggregation and charts over large data, and a recategorisation flow that feels instant yet stays consistent
P06	BillBuddy — Bill payments, recharges and autopay	Payments	Intermediate	Server-driven dynamic forms, and many independent external systems that can each fail on their own
P07	BranchQ — Branch appointments and live queue tokens	Branch services	Beginner–Intermediate	Time-based, contended resources (slots and queues) with honest real-time updates that do not drain the battery
P08	OpenAccount — Digital savings account opening with KYC	Onboarding	Advanced	A long, regulated, interruptible state machine with sensitive data at every step
P09	InvestLite — Mutual funds and SIP tracker	Wealth	Advanced	Heavy charts and numeric correctness (XIRR, units, cut-offs), kept fast and honest
P10	FraudShield — Real-time transaction alerts and dispute management	Risk & servicing	Advanced	Real-time streams, safety-critical actions that must be instant and idempotent, and a branching dispute flow with an auditable trail

Device capabilities per project

Project	Device features used
P01 PayLite	Camera · Biometrics · Contacts · Push notifications
P02 CardVault	Biometrics · Secure screen · Push notifications
P03 LoanEase	Camera · File picker · Biometrics
P04 SaveSmart	Local notifications · Biometrics
P05 SpendWise	Notifications · Share sheet

Project	Device features used
P06 BillBuddy	Notifications · Contacts (recharge) · Share sheet
P07 BranchQ	GPS · Maps · Notifications
P08 OpenAccount	Camera · SMS retriever (OTP) · Biometrics
P09 InvestLite	Biometrics · Notifications
P10 FraudShield	Push notifications · Biometrics · File picker

What every project must demonstrate

The capstone is assessed against the course, not just the feature list. Each module maps to concrete evidence in the project:

Module	Required evidence
M1 · Dart & Flutter basics	Null-safe models; money as <code>int</code> paise; no <code>dynamic</code> outside <code>fromJson</code> ; const constructors.
M2 · UI & navigation	Responsive layouts without overflow at 200% text scale; forms with validators; <code>go_router</code> with nested routes, path/query parameters and results returned via <code>pop</code> .
M3 · State management	Riverpod throughout; <code>ref.watch</code> in build and <code>ref.read</code> in handlers; immutable state; a router guard driven by session state.
M4 · API integration	One repository per domain; Dio with auth and 401 interceptors; one sealed error type; <code>AsyncValue.when</code> with loading, error and data on every screen; idempotent money movements.
M5 · Performance, motion, security	Profiled in profile mode with before/after numbers; skeletons; at least two purposeful animations honouring reduce motion; token in secure storage; biometric lock; <code>FLAG_SECURE</code> on sensitive screens; no secrets in logs.
M6 · Testing & CI/CD	Unit tests for business rules, widget tests for key screens, one integration test for the main journey; $\geq 70\%$ coverage; GitHub Actions running format, analyze and tests on every pull request.
M7 · Capstone	Working product demo, architecture walkthrough, and a short retrospective.

Baseline functional requirements (all projects)

These are not repeated inside each project, but they are part of every project's Must scope.

#	Requirement	Detail
B1	Authentication	Login (or onboarding), session restored from secure storage, logout from the profile screen.
B2	App lock	Lock on background; biometric unlock with device-PIN fallback; "Log out instead" option.
B3	Route guard	Deep links while logged out land on login; logout needs no navigation code.
B4	Three states everywhere	Every network-backed screen has designed loading (skeleton), error (with retry) and empty states.
B5	Refresh	Pull-to-refresh on every list; data invalidated after any mutation that changes it.
B6	Idempotency	Every call that moves money or creates a record sends an <code>Idempotency-Key</code> created once per confirmation screen.
B7	Errors in plain language	No stack traces, HTML or raw codes on screen; server messages shown when they exist.
B8	Accessibility	Tooltips on icon buttons; text alternatives for colour; usable at 200% font scale; reduce motion honoured.
B9	Environments	<code>--dart-define</code> for dev/staging/prod API URLs; no URL hard-coded in screens.

Baseline non-functional requirements

Quality	Requirement	How it is verified
Performance	Scrolling lists at 60 fps in profile mode on a mid-range Android device; no frame over 32 ms during normal use.	DevTools screenshots in the PR
Start-up	Cold start to first meaningful screen under 2.5 s (release build).	Stopwatch + DevTools timeline
Security	Tokens only in Keychain/Keystore; FLAG_SECURE on money and identity screens; release build obfuscated; no tokens, PINs or account numbers in logs.	Code review + log inspection
Reliability	No duplicate money movement under retry; graceful behaviour with no network, slow network and server errors.	Chaos switches on the mock API
Accessibility	WCAG-inspired checks: labels, contrast, text scale, reduce motion.	Manual checklist
Maintainability	Feature-first folders; no widget imports Dio; <code>flutter analyze</code> clean; formatted.	CI
Testability	≥ 70% line coverage; business rules fully unit-tested; one integration test.	CI coverage gate
Observability	Server traceId surfaced in debug logs; errors categorised by BankError type.	Code review

Reference architecture

Every project uses the same four client layers. Only the contents change.

Reference architecture (all projects)

Flutter client · Riverpod · Repository pattern · REST over HTTPS

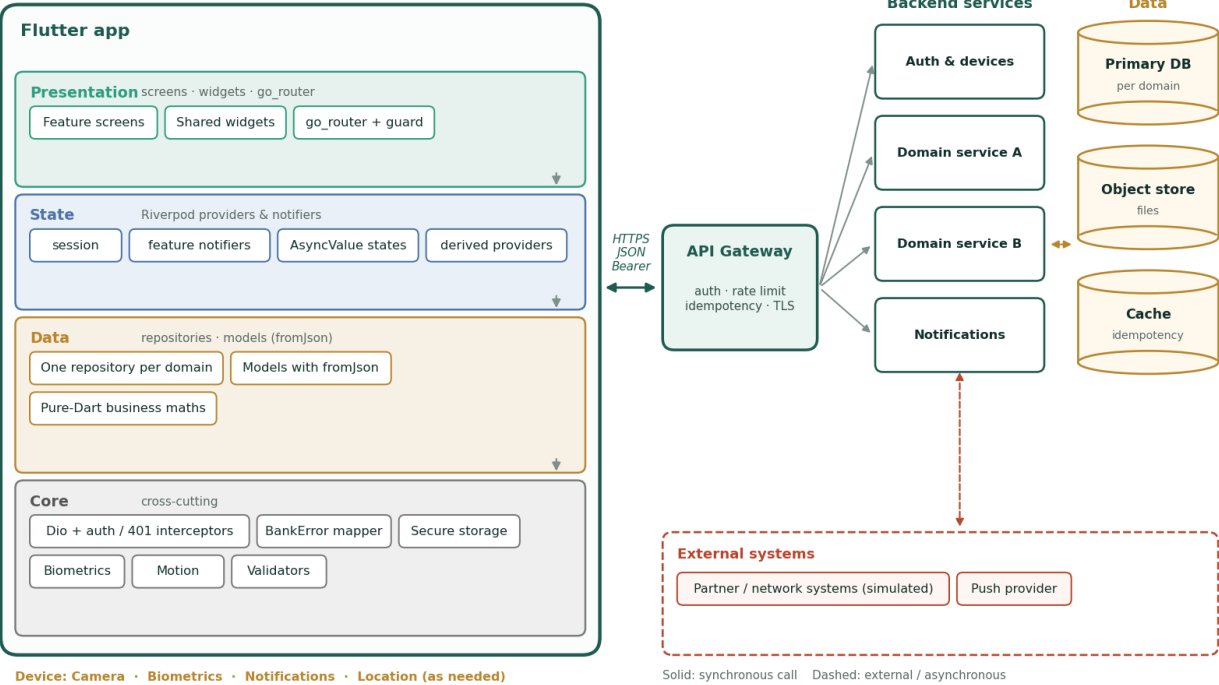


Figure A · Reference architecture shared by all ten projects

Layer	Responsibility
Presentation	Screens and widgets; routing with go_router and a session-driven guard. Knows nothing about HTTP.
State	Riverpod providers and notifiers holding AsyncValues. Screens watch; handlers read.
Data	One repository per backend domain. Converts JSON to models at the border and DioException to BankError. Pure-Dart business maths lives here and is unit-tested.
Core	Shared, feature-agnostic code: the Dio client and interceptors, the error mapper, secure storage, biometrics, motion, validators.
API gateway	Authentication, rate limiting, idempotency and TLS termination. In class, the provided mock server plays this role.
Services & data	Simulated by the mock backend. The app must not assume anything about them beyond the API contract.

Folder structure template

```

lib/
├── main.dart
├── app/
│   ├── app.dart · router.dart · routes.dart · theme.dart
│   └── core/
│       ├── errors/      bank_error.dart
│       ├── network/     api_config.dart · api_client.dart · error_mapper.dart
│       ├── security/    secure_session_store.dart · biometric_service.dart · app_lock.dart
│       ├── motion/      motion.dart · ...
│       ├── utils/       money.dart · date_format.dart · validators.dart
│       └── widgets/     async_error_view.dart · skeleton.dart · ...
└── features/
    └── <feature>/
        ├── domain/      models (fromJson)
        ├── data/         <feature>_repository.dart
        ├── state/        <feature>_provider.dart
        ├── presentation/ screens
        └── widgets/      feature-only widgets
test/ unit/ · widget/ · helpers/ (fake API, test app, flows)
integration_test/

```

API conventions (all projects)

Topic	Convention
Money	Integer paise in JSON (<code>"amountPaise": 50000</code>). Never floats, never formatted strings.
Dates	ISO-8601 UTC strings; the app converts to local time once, in the model.
Errors	<code>{ "error": { "code", "message", "details", "traceId" } }</code> with 400/401/403/404/409/422/5xx semantics as taught in Module 4.
Idempotency	<code>Idempotency-Key</code> header on every mutating call that moves money or creates a record; same key → same response.
Paging	Cursor paging (<code>?cursor=&limit=</code>) returning <code>{ items, nextCursor }</code> .
Auth	<code>Authorization: Bearer <token></code> ; 401 means sign in again.

Backend options

Option	When to use it
A · Mock Node API (recommended)	Extend the BankEase API: in-memory data, the error shape, idempotency and the chaos switches already exist. Fastest path and fully controllable in class.
B · Firebase	Firebase Auth for identity, Firestore for data with owner-scoped read rules, and a small trusted server (or Cloud Functions) for every money movement — the pattern from the BankEase Firebase edition.
C · json-server	Quick start for UI-heavy projects (BranchQ, SpendWise). Idempotency and error shapes must then be simulated in a fake adapter for tests.

Milestones, deliverables and evaluation

Milestone	Deliverables	Modules
1 · Foundations	Folder structure, theme, router with guard, login, models with fromJson + unit tests	M1–M3
2 · Data	Repositories, error mapping, all read screens with loading/error/empty states	M4
3 · Money & forms	All Must write flows with validation and idempotency	M2–M4
4 · Quality	Profiling numbers, skeletons, motion, secure storage, biometric lock, FLAG_SECURE	M5
5 · Ship	Tests to ≥ 70%, CI green, signed release build, demo and retrospective	M6–M7

Submission checklist

- Git repository with a protected `main`, pull-request history, and CI badges green.
- README: setup, how to run the mock API, environment variables, architecture diagram, and known limitations.
- A 5-minute demo video of the main journey, including one failure path (no network or server error).
- Profiling evidence: before/after worst-frame numbers for one list screen.
- Signed release build (APK or AAB) with symbols kept.

Evaluation rubric (100 points)

Area	What earns the points	Points
Functional completeness	All Must features working; Should features attempted	25
Architecture & code quality	Layers respected; no Dio in widgets; readable, formatted, analyzer-clean	15
State management	Correct watch/read; immutable state; no stale data after mutations	10
API & error handling	Repositories, BankError, three states per screen, idempotency	15
Performance & motion	Measured improvements; purposeful animations; reduce motion	10
Security	Secure storage, biometric lock, FLAG_SECURE, clean logs, obfuscated release	10
Testing & CI/CD	Coverage, meaningful tests, pipeline, protected main	10
Demo & communication	Clear demo, honest retrospective, architecture explained	5

P01 · PayLite

UPI-style person-to-person payments with QR scan-and-pay

Theme	Payments
Level	Advanced
Device features	Camera · Biometrics · Contacts · Push notifications
The hard part	Payment status is not binary. The hard part is modelling SUCCESS / PENDING / FAILED honestly, polling without leaking timers, and making retries safe.

1. Problem statement

Context. Digital payments in India are dominated by UPI: a customer pays another person or a shop by scanning a QR code or entering a UPI ID (VPA), and the money moves instantly. A mid-sized bank wants its own lightweight payments app so customers stop leaving for third-party wallets.

Problem. Customers today must open the full net-banking app, add a beneficiary and wait for activation before paying someone new. Small payments — splitting a dinner bill, paying a vegetable seller — are too slow, and failed or duplicate payments generate a high volume of support calls.

Goal. Build PayLite, a Flutter app that lets a signed-in customer pay any VPA or QR code in under ten seconds, request money from others, split a bill between friends, and see every payment's status in real time — without ever charging twice.

2. Users

Persona	Situation	What they need
Priya, 24, software engineer	Splits rent and dinners with flatmates weekly	Fast payments and one-tap requests
Ramesh, 48, vegetable vendor	Receives dozens of small payments daily	A clear "received" confirmation he can trust
Asha, 62, retired teacher	Pays occasionally, anxious about mistakes	Large text, a confirm step, and a visible PIN step

3. Features

10 features: 7 Must · 2 Should · 1 Could — in addition to baseline requirements B1–B9.

ID	Feature	Description	Modules	Priority
F1	Sign in and device binding	Login with customer ID + PIN; the device is bound on first login and the session is restored from secure storage with biometric unlock.	M4, M5	Must
F2	Home and balance	Primary account balance (hidden by default), recent payments, and quick actions: Scan, Pay, Request, Split.	M2, M3	Must
F3	Scan and pay	Scan a UPI QR code with the camera, parse payee name, VPA and optional amount, then go to the pay screen pre-filled.	M2, M5	Must

ID	Feature	Description	Modules	Priority
F4	Pay to VPA	Enter a VPA, verify it with the server (shows the registered name), enter amount and note, review, and confirm with the UPI PIN.	M2, M4	Must
F5	Idempotent payment	Every payment attempt carries an Idempotency-Key created once per review screen; a timeout followed by retry never debits twice.	M4, M6	Must
F6	Payment status	Payments can be SUCCESS, PENDING or FAILED; pending payments are polled every 5 s for up to 2 minutes, then shown as "Pending — we will notify you".	M3, M4	Must
F7	Request money	Send a collect request to a VPA; the receiver sees it in "Requests" and can pay or decline.	M3, M4	Should
F8	Split bill	Pick 2–10 contacts, split equally or by custom amounts that must add up to the total, and send one collect request each.	M2, M3	Should
F9	History and search	Infinite-scroll payment history with filters (sent, received, failed) and search by name or VPA.	M2, M4, M5	Must
F10	Receipt and share	A receipt screen with reference number, time and status, shareable as text.	M2	Could

4. Acceptance criteria

F1 · Sign in and device binding (Must)

- A second device cannot use a session token issued to the first
- Reopening the app requires biometrics before any balance is shown

F2 · Home and balance (Must)

- Balance is hidden until the eye is tapped
- Pull to refresh re-fetches balance and recent payments

F3 · Scan and pay (Must)

- A malformed QR shows "This is not a valid payment QR" and never navigates
- A QR with a fixed amount locks the amount field

F4 · Pay to VPA (Must)

- Unknown VPAs show "No account found for this UPI ID" before the amount step
- Amounts above ₹1,00,000 are blocked with the limit message

F5 · Idempotent payment (Must)

- A retry after a 30-second timeout returns the original receipt
- A unit test proves one key produces exactly one debit

F6 · Payment status (Must)

- A pending payment turns into success on screen without the user refreshing
- Polling stops when the screen is closed

F7 · Request money (Should)

- A declined request shows "Declined" on the requester's side within one refresh
- Expired requests (48 h) cannot be paid

F8 · Split bill (Should)

- Custom amounts that do not add up disable Send and show the difference

- Each participant's status (paid/pending) is visible

F9 · History and search (Must)

- 500+ rows scroll at 60 fps in profile mode
- Filters and search combine

F10 · Receipt and share (Could)

- The UPI reference number is copyable with one tap

5. Screens and navigation

Screen	Route	Purpose
Login	/login	Customer ID + PIN, then device binding
Lock	overlay	Biometric unlock on resume
Home	/home	Balance, quick actions, recent payments
Scan	/scan	Camera QR scanner
Pay	/pay?vpa=...	VPA, verified name, amount, note
Review	/pay/review	Summary and confirm
UPI PIN	/pay/pin	4–6 digit PIN pad
Status	/pay/status/:id	Success / pending / failed
Requests	/requests	Incoming and outgoing collect requests
Split	/split	Participants and amounts
History	/history	Filters, search, infinite scroll
Receipt	/history/:id	Shareable receipt

6. Data model

Entity	Fields
Account	id, maskedNumber, balancePaise, primaryVpa
Vpa	address, verifiedName, bankName
Payment	id, direction, counterparty, amountPaise, note, status, upiRef, createdAt
CollectRequest	id, from, to, amountPaise, status, expiresAt
SplitGroup	id, totalPaise, participants[]

All money fields are integer paise; all timestamps are ISO-8601 UTC.

7. API contract

Method	Path	Purpose	Notable errors
POST	/auth/login	Customer ID + PIN → token + device id	401, 423 locked
GET	/accounts/primary	Balance and primary VPA	401
GET	/vpa/{address}	Verify a VPA, return the registered name	404 VPA_NOT_FOUND

Method	Path	Purpose	Notable errors
POST	/payments	Pay (Idempotency-Key header, UPI PIN hash)	422 INSUFFICIENT_FUNDS, 422 LIMIT_EXCEEDED, 409 KEY_REUSED
GET	/payments/{id}	Poll a payment status	404
GET	/payments?cursor=&filter=&q=	History with cursor paging	—
POST	/collect-requests	Request money	404, 422
POST	/collect-requests/{id}/pay /decline	Act on a request	410 EXPIRED

8. Non-functional requirements

Quality	Requirement (in addition to the baseline)
Performance	Pay flow from Scan to Status in under 10 s on a mid-range phone; history scrolls at 60 fps with 500 rows (profile mode)
Reliability	Zero duplicate debits under retry; pending payments resolve or are reported within 2 minutes
Security	Token and device id in Keychain/Keystore; UPI PIN never stored or logged, sent only as a hash; FLAG_SECURE on pay and PIN screens
Availability	If the server is down, the app shows a clear status and never shows a stale balance as current
Accessibility	PIN pad and amounts readable at 200% font scale; every icon button has a tooltip; reduce motion honoured
Usability	A first-time user can pay a QR in 3 taps after scanning
Testability	≥ 70% line coverage; idempotency and status polling covered by unit tests; one integration test from scan to status
Observability	Every error carries the server traceId in debug logs; no personal data in logs

9. Architecture

P01 · PayLife · Architecture

Flutter client · Riverpod · Repository pattern · REST over HTTPS

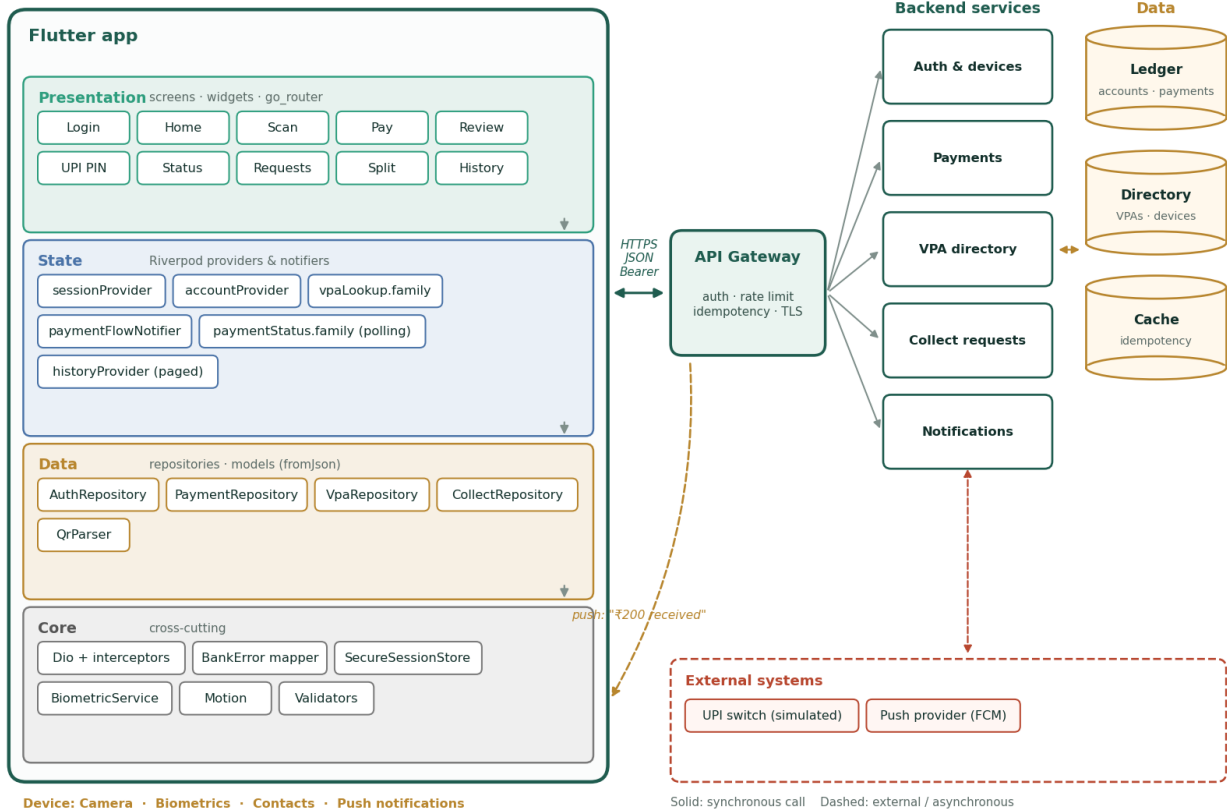


Figure P01 · PayLife architecture

- **Presentation:** 10 screens (Login, Home, Scan, Pay, Review, UPI PIN, Status, Requests, Split, History), all routed through go_router with the session guard.
- **State:** sessionProvider, accountProvider, vpaLookup.family, paymentFlowNotifier, paymentStatus.family (polling), historyProvider (paged). Screens *watch* these; buttons *read* their notifiers.
- **Data:** AuthRepository, PaymentRepository, VpaRepository, CollectRepository, QrParser — each repository owns one backend domain and throws only *BankError*.
- **Core:** Dio + interceptors, BankError mapper, SecureSessionStore, BiometricService, Motion, Validators.
- **Backend (simulated):** Auth & devices, Payments, VPA directory, Collect requests, Notifications, over Ledger, Directory, Cache.
- **External systems (simulated):** UPI switch (simulated), Push provider (FCM). Treat them as unreliable: they can be slow, down, or return partial results.
- **Push:** push: "₹200 received" — the notification opens the related screen through a deep link.

10. Edge cases the solution must handle

1. The camera permission is denied: show why it is needed and a button to open settings
2. A QR encodes a VPA with a trailing space or lowercase letters: normalise before verifying
3. The network drops after the PIN is entered: show Pending, never "Failed", until the server confirms
4. The user double-taps Confirm: only one request leaves the device
5. A collect request expires while the user is on its screen: disable Pay and explain
6. Split amounts in paise do not divide evenly (₹100 / 3): assign the extra paise to the first participant

11. Stretch goals

- Offline QR "pay later" queue that sends when back online, with the same idempotency key
- Home-screen widget showing the last 3 payments
- Merchant mode with a static QR and a daily collection summary

12. Definition of done

Area	Done when...
Features	All Must features and baseline B1–B9 working against the mock API
The hard part	Payment status is not binary. The hard part is modelling SUCCESS / PENDING / FAILED honestly, polling without leaking timers, and making retries safe.
Tests	Business rules unit-tested; widget tests for every Must screen; one integration test for the main journey; $\geq 70\%$ coverage
Quality	Profiling numbers in the PR; skeletons; reduce motion; FLAG_SECURE where money or identity is shown
Delivery	CI green on protected main; signed, obfuscated release build; README with setup and this architecture

P02 · CardVault

Debit and credit card control centre

Theme	Cards
Level	Intermediate
Device features	Biometrics · Secure screen · Push notifications
The hard part	Optimistic UI done right: instant toggles that stay truthful when the network fails, plus a reveal screen that is genuinely secure.

1. Problem statement

Context. Card fraud and lost cards are the most common reasons customers call a bank. Today, freezing a card or changing its limits means a phone call and a 20-minute wait.

Problem. Customers cannot see all their cards in one place, cannot temporarily freeze a card they have misplaced, and cannot turn off international or online use when they do not need it. Every change is a support ticket.

Goal. Build CardVault, a Flutter app where customers see all their debit and credit cards, freeze and unfreeze them instantly, set spending limits and channel controls, view credit card statements and dues, and reveal card details securely.

2. Users

Persona	Situation	What they need
Kiran, 31, frequent traveller	Uses cards abroad a few times a year	Turn international use on only when travelling
Meena, 40, small-business owner	Holds three credit cards	Track dues and avoid late fees
Arjun, 22, student	First debit card, often misplaces it	Freeze first, search the bag later

3. Features

10 features: 7 Must · 2 Should · 1 Could — in addition to baseline requirements B1–B9.

ID	Feature	Description	Modules	Priority
F1	Card wallet	All cards as a horizontal carousel of card visuals (network, masked number, expiry, status badge).	M2, M5	Must
F2	Freeze / unfreeze	Instantly freeze or unfreeze a card with a confirmation sheet; the change is optimistic and rolls back on failure.	M3, M4	Must
F3	Channel controls	Toggle online, contactless, ATM and international usage per card.	M2, M3, M4	Must
F4	Spending limits	Set daily limits per channel with a slider and a text field; limits cannot exceed the bank maximum.	M2, M3	Must
F5	Reveal card details	Show full number, expiry and CVV for 30 seconds after biometric check, with copy buttons, then auto-hide.	M5	Must
F6	Credit card summary	Credit limit, available limit, current outstanding, minimum due and due date, with a progress ring.	M2, M4	Must

ID	Feature	Description	Modules	Priority
F7	Statements	Monthly statements list, statement detail with transactions grouped by date, and category totals.	M2, M4, M5	Should
F8	Pay credit card bill	Pay minimum, total or custom amount from a savings account (idempotent).	M4	Should
F9	Report lost / stolen	Permanently block a card with a reason and request a replacement; irreversible, with a typed confirmation.	M2, M4	Must
F10	Transaction alerts	A notifications list of recent card transactions with merchant and location.	M3, M4	Could

4. Acceptance criteria

F1 · Card wallet (Must)

- Frozen cards show a frost overlay and a "Frozen" badge
- The carousel animates between cards with page snapping

F2 · Freeze / unfreeze (Must)

- The toggle updates immediately and reverts with an error message if the server rejects it
- Freezing requires biometric confirmation

F3 · Channel controls (Must)

- Each toggle saves independently; a failed save reverts only that toggle
- International can be enabled with an end date (auto-off)

F4 · Spending limits (Must)

- The slider and text field stay in sync
- Values above the maximum are clamped with a message

F5 · Reveal card details (Must)

- Details auto-hide after 30 s or when the app goes to background
- Screenshots are blocked on this screen

F6 · Credit card summary (Must)

- Due within 3 days shows a warning banner
- Amounts are formatted in Indian grouping

F7 · Statements (Should)

- Switching months keeps scroll position per month
- Large statements stay at 60 fps

F8 · Pay credit card bill (Should)

- Paying twice with the same key debits once

F9 · Report lost / stolen (Must)

- The user must type BLOCK to confirm
- A blocked card cannot be unfrozen

F10 · Transaction alerts (Could)

- Tapping an alert opens the transaction detail

5. Screens and navigation

Screen	Route	Purpose
Login	/login	Sign in
Wallet	/cards	Card carousel and quick controls
Card detail	/cards/:id	Controls, limits, recent transactions
Limits	/cards/:id/limits	Per-channel sliders
Reveal	/cards/:id/reveal	Secure details (30 s)
Credit summary	/cards/:id/credit	Dues and ring
Statements	/cards/:id/statements	Month list
Statement	/cards/:id/statements/:month	Transactions by date
Pay bill	/cards/:id/pay	Amount options
Block	/cards/:id/block	Lost / stolen flow

6. Data model

Entity	Fields
Card	id, type, network, maskedNumber, expiry, status (ACTIVE/FROZEN/BLOCKED)
Controls	online, contactless, atm, international, internationalUntil
Limits	atmDailyPaise, posDailyPaise, onlineDailyPaise, maxPaise
CreditSummary	limitPaise, availablePaise, outstandingPaise, minDuePaise, dueDate
Statement	month, openingPaise, closingPaise, transactions[]

All money fields are integer paise; all timestamps are ISO-8601 UTC.

7. API contract

Method	Path	Purpose	Notable errors
GET	/cards	All cards	401
PATCH	/cards/{id}/status	FREEZE / UNFREEZE	409 BLOCKED
PUT	/cards/{id}/controls	Channel toggles	422
PUT	/cards/{id}/limits	Daily limits	422 ABOVE_MAX
POST	/cards/{id}/reveal	One-time sensitive details (short-lived token)	403
GET	/cards/{id}/credit-summary	Dues	—
GET	/cards/{id}/statements	Months	—
GET	/cards/{id}/statements/{month}	Transactions	404
POST	/cards/{id}/payments	Pay bill (Idempotency-Key)	422
POST	/cards/{id}/block	Permanent block	409

8. Non-functional requirements

Quality	Requirement (in addition to the baseline)
Performance	Freeze toggle feels instant (optimistic UI, < 100 ms visual response); carousel at 60 fps
Security	Card number, expiry and CVV never stored on the device, never logged, never in a screenshot; revealed data cleared from memory on hide
Consistency	Optimistic updates always reconcile with the server; a failed save reverts and explains
Accessibility	Card visuals have semantic labels ("Visa debit ending 4417, frozen"); toggles announce state
Usability	Freeze reachable in at most 2 taps from launch
Testability	Optimistic rollback covered by unit tests; widget tests for every control; ≥ 70% coverage
Reliability	Duplicate bill payments impossible (idempotency)
Compliance	Show last 4 digits only, everywhere except the reveal screen (PCI DSS display rule)

9. Architecture

P02 · CardVault · Architecture

Flutter client · Riverpod · Repository pattern · REST over HTTPS

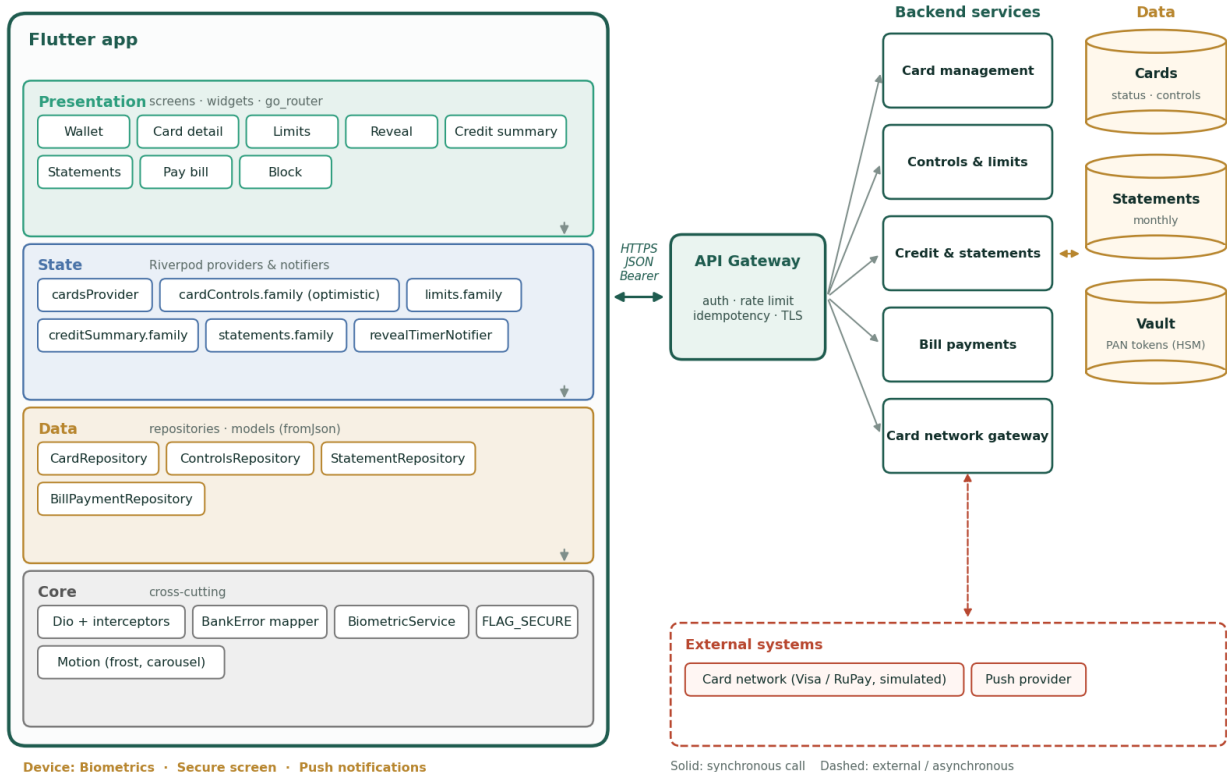


Figure P02 · CardVault architecture

- **Presentation:** 8 screens (Wallet, Card detail, Limits, Reveal, Credit summary, Statements, Pay bill, Block), all routed through go_router with the session guard.
- **State:** cardsProvider, cardControls.family (optimistic), limits.family, creditSummary.family, statements.family, revealTimerNotifier. Screens **watch** these; buttons **read** their notifiers.
- **Data:** CardRepository, ControlsRepository, StatementRepository, BillPaymentRepository — each repository owns one backend domain and throws only **BankError**.
- **Core:** Dio + interceptors, BankError mapper, BiometricService, FLAG_SECURE, Motion (frost, carousel).

- **Backend (simulated):** Card management, Controls & limits, Credit & statements, Bill payments, Card network gateway, over Cards, Statements, Vault.
- **External systems (simulated):** Card network (Visa / RuPay, simulated), Push provider. Treat them as unreliable: they can be slow, down, or return partial results.

10. Edge cases the solution must handle

1. Freeze succeeds on the server but the response is lost: the next refresh must show the true state
2. Two quick taps on the same toggle: the last intent wins, and only one request per toggle is in flight
3. The reveal timer is running when a phone call arrives: details are hidden immediately
4. International auto-off date is in the past: reject with a clear message
5. A blocked card appears in the wallet: controls are hidden, only "Replacement status" is shown
6. Statement for a month with no transactions: show an empty state, not an error

11. Stretch goals

- Virtual card generation for one-time online use
- Merchant category blocking (gambling, crypto)
- Card-level spend analytics chart

12. Definition of done

Area	Done when...
Features	All Must features and baseline B1–B9 working against the mock API
The hard part	Optimistic UI done right: instant toggles that stay truthful when the network fails, plus a reveal screen that is genuinely secure.
Tests	Business rules unit-tested; widget tests for every Must screen; one integration test for the main journey; $\geq 70\%$ coverage
Quality	Profiling numbers in the PR; skeletons; reduce motion; FLAG_SECURE where money or identity is shown
Delivery	CI green on protected main; signed, obfuscated release build; README with setup and this architecture

P03 · LoanEase

Personal loan application and EMI tracker

Theme	Lending
Level	Advanced
Device features	Camera · File picker · Biometrics
The hard part	A long, interruptible workflow: resumable drafts, reliable uploads with progress, and money maths that are provably correct.

1. Problem statement

Context. Personal loan applications still involve paper forms, branch visits and a week of back-and-forth for missing documents. Most drop-offs happen at the documents step.

Problem. Applicants do not know whether they are eligible until after applying, cannot see where their application is stuck, and existing borrowers have no clear view of their EMI schedule or the effect of a prepayment.

Goal. Build LoanEase: an eligibility checker, a multi-step application with document upload and resumable drafts, live application tracking, and an EMI dashboard with a prepayment simulator.

2. Users

Persona	Situation	What they need
Sunil, 29, salaried	Needs ₹3 lakh for a wedding	Know eligibility and EMI before applying
Farah, 35, self-employed	Irregular income, many documents	Save progress and finish later
Vikram, 45, existing borrower	Paying an EMI for 2 years	See how a prepayment shortens the loan

3. Features

10 features: 7 Must · 2 Should · 1 Could — in addition to baseline requirements B1–B9.

ID	Feature	Description	Modules	Priority
F1	EMI calculator	Amount, tenure and rate sliders with live EMI, total interest and a principal/interest chart.	M2, M3	Must
F2	Eligibility check	Income, obligations and employment type → server returns eligible amount and indicative rate.	M2, M4	Must
F3	Multi-step application	5 steps: personal, employment, loan details, documents, review — with a progress stepper and per-step validation.	M2, M3	Must
F4	Resumable draft	Each step is saved as a server draft; reopening the app resumes at the last incomplete step.	M3, M4	Must
F5	Document upload	Capture or pick PAN, Aadhaar (masked), salary slips and bank statement; show upload progress, retry and replace.	M4, M5	Must
F6	Submit and consent	Review screen with all details, explicit consent checkboxes and a final submit (idempotent).	M2, M4	Must

ID	Feature	Description	Modules	Priority
F7	Application tracker	Timeline: Submitted → Documents verified → Credit check → Approved/Rejected → Disbursed, with timestamps.	M2, M3, M4	Must
F8	Loan dashboard	Active loans with outstanding principal, next EMI date and a paid/remaining progress bar.	M2, M4	Should
F9	EMI schedule	Full amortisation table (month, EMI, principal, interest, balance) with infinite scroll.	M2, M5	Should
F10	Prepayment simulator	Enter a prepayment → compare "reduce tenure" vs "reduce EMI" side by side, with interest saved.	M3	Could

4. Acceptance criteria

F1 · EMI calculator (Must)

- EMI matches the standard formula to the paise (unit-tested)
- Changing any slider updates results within one frame

F2 · Eligibility check (Must)

- Soft check: no credit-score impact message shown
- Ineligible users see the reason and a smaller suggested amount

F3 · Multi-step application (Must)

- The Next button is disabled until the step is valid
- Back never loses entered data

F4 · Resumable draft (Must)

- Killing the app on step 3 and reopening resumes on step 3 with data
- Drafts expire after 30 days with a warning

F5 · Document upload (Must)

- Files over 5 MB are compressed or rejected with a message
- A failed upload can be retried without re-selecting the file

F6 · Submit and consent (Must)

- Submit is disabled until both consents are ticked
- Double submit creates one application

F7 · Application tracker (Must)

- Status refreshes on pull and on resume
- Rejected shows the reason category

F8 · Loan dashboard (Should)

- Overdue EMIs show a red banner with the amount due

F9 · EMI schedule (Should)

- 360-row schedules scroll smoothly

F10 · Prepayment simulator (Could)

- Results computed locally and match the server within ₹1

5. Screens and navigation

Screen	Route	Purpose
Home	/home	Offers and active loans
Calculator	/calculator	Sliders and chart
Eligibility	/eligibility	Soft check
Apply (stepper)	/apply/:step	5 steps
Documents	/apply/documents	Upload list
Review & consent	/apply/review	Submit
Tracker	/applications/:id	Timeline
Loan	/loans/:id	Dashboard
Schedule	/loans/:id/schedule	Amortisation
Prepay	/loans/:id/prepay	Simulator

6. Data model

Entity	Fields
Application	id, status, currentStep, draft{...}, createdAt, updatedAt
Document	id, type, fileName, sizeBytes, status (UPLOADING/UPLOADED/REJECTED)
StatusEvent	status, at, note
Loan	id, principalPaise, ratePct, tenureMonths, outstandingPaise, nextEmiDate
Installment	n, dueDate, emiPaise, principalPaise, interestPaise, balancePaise, paid

All money fields are integer paise; all timestamps are ISO-8601 UTC.

7. API contract

Method	Path	Purpose	Notable errors
POST	/eligibility	Soft eligibility check	422
POST	/applications	Create draft	—
PATCH	/applications/{id}	Save a step	422 field errors
POST	/applications/{id}/documents	Multipart upload with progress	413 TOO_LARGE, 415 TYPE
POST	/applications/{id}/submit	Submit (Idempotency-Key)	409, 422 INCOMPLETE
GET	/applications/{id}	Status + timeline	404
GET	/loans	Active loans	—
GET	/loans/{id}/schedule	Amortisation	—
POST	/loans/{id}/prepayment-quote	Server quote	422

8. Non-functional requirements

Quality	Requirement (in addition to the baseline)
Correctness	EMI and amortisation match the standard reducing-balance formula; money in paise, rounding rules documented and tested
Resilience	No data lost if the app is killed mid-application; uploads resume or retry cleanly
Performance	Calculator updates within one frame; 360-row schedule at 60 fps
Security	Documents never cached in shared storage; Aadhaar shown masked (XXXX XXXX 1234); consent timestamps recorded server-side
Accessibility	Stepper announces "Step 2 of 5"; sliders have text-field alternatives
Usability	Application completable in under 8 minutes with documents at hand
Testability	Unit tests for EMI maths and step validation; widget test for the full stepper; ≥ 75% coverage
Data minimisation	Only fields required for the loan type are collected

9. Architecture

P03 · LoanEase · Architecture

Flutter client · Riverpod · Repository pattern · REST over HTTPS

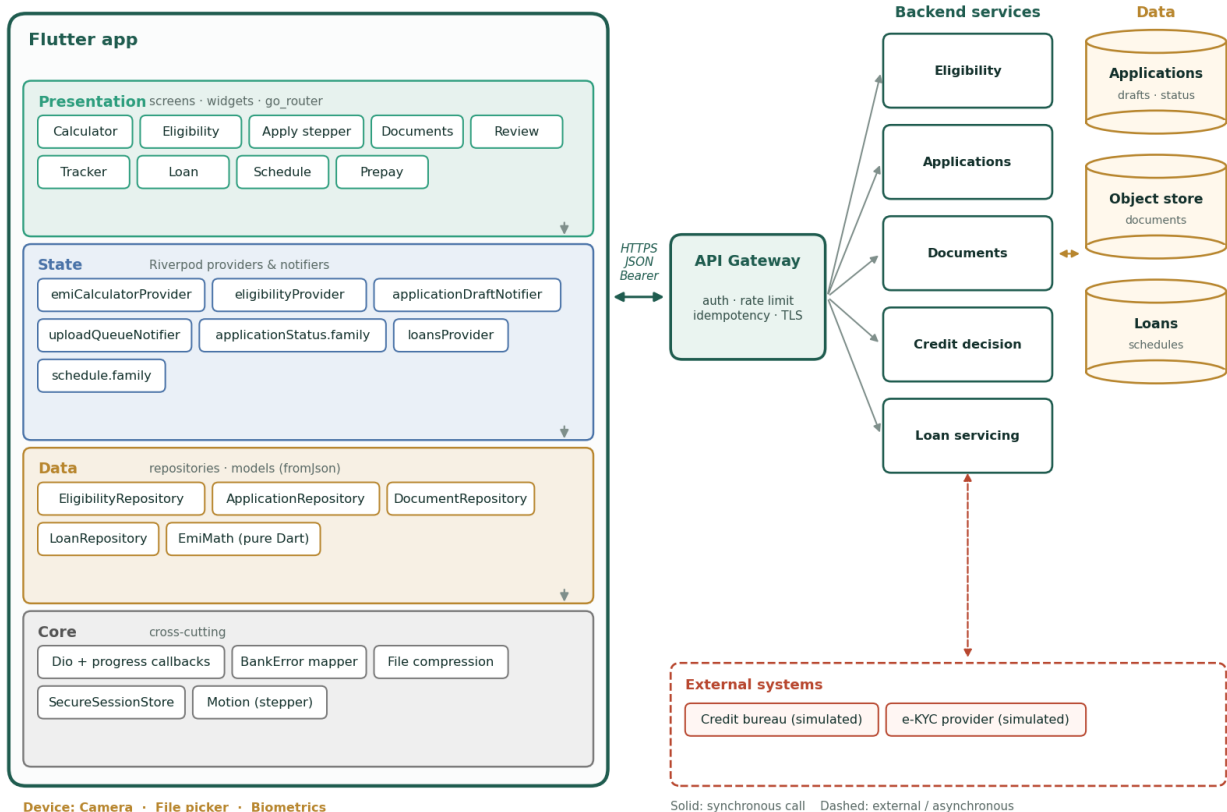


Figure P03 · LoanEase architecture

- **Presentation:** 9 screens (Calculator, Eligibility, Apply stepper, Documents, Review, Tracker, Loan, Schedule, Prepay), all routed through go_router with the session guard.
- **State:** emiCalculatorProvider, eligibilityProvider, applicationDraftNotifier, uploadQueueNotifier, applicationStatus.family, loansProvider, schedule.family. Screens *watch* these; buttons *read* their notifiers.

- **Data:** EligibilityRepository, ApplicationRepository, DocumentRepository, LoanRepository, EmiMath (pure Dart) — each repository owns one backend domain and throws only `BankError`.
- **Core:** Dio + progress callbacks, BankError mapper, File compression, SecureSessionStore, Motion (stepper).
- **Backend (simulated):** Eligibility, Applications, Documents, Credit decision, Loan servicing, over Applications, Object store, Loans.
- **External systems (simulated):** Credit bureau (simulated), e-KYC provider (simulated). Treat them as unreliable: they can be slow, down, or return partial results.

10. Edge cases the solution must handle

1. Upload succeeds but the app is killed before the response: the document list must reconcile on resume
2. Two devices edit the same draft: last save wins with a "changed elsewhere" notice
3. Rate changes between calculator and submit: the review screen shows the final rate prominently
4. A rejected document (blurry): show which one and allow replacing only that file
5. Tenure slider at maximum with tiny amount: EMI rounding must never produce negative interest
6. Consent withdrawn before decision: application moves to Withdrawn

11. Stretch goals

- Auto-fill from a salary slip photo (OCR)
- Push notifications on every status change
- Pre-approved offers for existing customers

12. Definition of done

Area	Done when...
Features	All Must features and baseline B1–B9 working against the mock API
The hard part	A long, interruptible workflow: resumable drafts, reliable uploads with progress, and money maths that are provably correct.
Tests	Business rules unit-tested; widget tests for every Must screen; one integration test for the main journey; $\geq 70\%$ coverage
Quality	Profiling numbers in the PR; skeletons; reduce motion; FLAG_SECURE where money or identity is shown
Delivery	CI green on protected main; signed, obfuscated release build; README with setup and this architecture

P04 · SaveSmart

Goal-based savings, recurring and fixed deposits

Theme	Savings
Level	Intermediate
Device features	Local notifications · Biometrics
The hard part	Financial maths that must be exactly right, plus motion that motivates without getting in the way.

1. Problem statement

Context. Customers keep large idle balances in savings accounts earning very little, while saying they want to save for goals such as a bike, a trip or an emergency fund.

Problem. Opening an FD or RD requires a branch visit or a hard-to-find menu, customers cannot compare maturity amounts, and nothing links a deposit to a goal they care about.

Goal. Build SaveSmart: goal creation with progress, automatic round-up and scheduled savings, FD and RD booking with maturity calculators, and a clear view of every deposit.

2. Users

Persona	Situation	What they need
Neha, 27, first job	Saving for a two-wheeler in 10 months	See progress and stay motivated
Rao, 55, pre-retiree	Moves money into FDs regularly	Compare tenures and book in one minute
Imran, 33, family man	Wants an emergency fund	Automatic monthly saving he can forget about

3. Features

10 features: 5 Must · 3 Should · 2 Could — in addition to baseline requirements B1–B9.

ID	Feature	Description	Modules	Priority
F1	Goals	Create a goal with name, target amount, target date and an icon; see a progress ring and required monthly saving.	M2, M3	Must
F2	Add money to a goal	One-off transfer from savings into a goal (idempotent) with a celebratory animation at milestones.	M4, M5	Must
F3	Auto-save rules	Round-up spare change, fixed weekly/monthly amount, or salary-day percentage; pause and resume.	M3, M4	Should
F4	FD calculator	Principal, tenure (7 days–10 years), payout type → maturity amount, interest and effective yield.	M2, M3	Must
F5	Book an FD	Choose account, amount, tenure, nominee and renewal instruction; review and confirm.	M2, M4	Must
F6	RD booking	Monthly instalment, tenure, debit date; shows the instalment schedule.	M2, M4	Should

ID	Feature	Description	Modules	Priority
F7	Deposits portfolio	All FDs and RDs with principal, rate, maturity date and a days-to-maturity bar; sort and filter.	M2, M4, M5	Must
F8	Premature withdrawal quote	Shows penalty-adjusted amount before breaking an FD; confirm with biometrics.	M4, M5	Should
F9	Nominee management	Add or change a nominee with relationship and share percentage totalling 100%.	M2	Could
F10	Maturity reminders	Local notification 7 days before maturity with a deep link.	M5	Could

4. Acceptance criteria

F1 · Goals (Must)

- Target dates in the past are rejected
- Required monthly amount recalculates when target or date changes

F2 · Add money to a goal (Must)

- Milestone animation at 25/50/75/100%, skipped with reduce motion
- A retry never moves money twice

F3 · Auto-save rules (Should)

- Paused rules show "Paused since ..."
- Rules cannot exceed 3 per goal

F4 · FD calculator (Must)

- Matches the quarterly compounding formula (unit-tested)
- Senior citizen toggle adds 0.5%

F5 · Book an FD (Must)

- Minimum ₹1,000 enforced
- Booking is idempotent

F6 · RD booking (Should)

- Debit date limited to 1–28

F7 · Deposits portfolio (Must)

- Sorting keeps the list at 60 fps with 200 deposits

F8 · Premature withdrawal quote (Should)

- The quote expires after 10 minutes

F9 · Nominee management (Could)

- Shares must add to 100%

F10 · Maturity reminders (Could)

- Tapping the notification opens that deposit

5. Screens and navigation

Screen	Route	Purpose
Home	/home	Goals and deposits summary
Goals	/goals	Goal cards
New goal	/goals/new	Form

Screen	Route	Purpose
Goal	/goals/:id	Progress, rules, history
Auto-save	/goals/:id/rules	Rules
FD calculator	/fd/calculator	Maturity maths
Book FD	/fd/new	Booking flow
RD	/rd/new	RD flow
Deposits	/deposits	Portfolio
Deposit	/deposits/:id	Detail and withdrawal quote

6. Data model

Entity	Fields
Goal	id, name, icon, targetPaise, savedPaise, targetDate, status
AutoSaveRule	id, goalId, type, amountPaise percent, schedule, paused
Deposit	id, kind (FD/RD), principalPaise, ratePct, startDate, maturityDate, maturityPaise, payout, renewal
Nominee	name, relation, sharePct

All money fields are integer paise; all timestamps are ISO-8601 UTC.

7. API contract

Method	Path	Purpose	Notable errors
GET/POST	/goals	List / create	422
POST	/goals/{id}/contributions	Add money (Idempotency-Key)	422 INSUFFICIENT_FUNDS
PUT	/goals/{id}/rules	Auto-save rules	422
GET	/deposits/rates	Rate card by tenure	—
POST	/deposits	Book FD/RD (Idempotency-Key)	422
GET	/deposits	Portfolio	—
POST	/deposits/{id}/withdrawal-quote	Premature quote	409
POST	/deposits/{id}/withdraw	Confirm with quote id	410 QUOTE_EXPIRED

8. Non-functional requirements

Quality	Requirement (in addition to the baseline)
Correctness	Maturity maths match published bank formulas to the rupee; rates always from the server, never hard-coded
Performance	Portfolio of 200 deposits sorts and scrolls at 60 fps
Security	Withdrawal requires biometric confirmation; FLAG_SECURE on booking review
Delight	Milestone animations under 1.2 s and never blocking; reduce motion disables them
Accessibility	Progress rings expose percentage as text

Quality	Requirement (in addition to the baseline)
Reliability	All money-moving calls idempotent
Testability	Calculator maths 100% unit-covered; $\geq 70\%$ overall
Offline	Goals and portfolio readable offline from the last sync, clearly labelled

9. Architecture

P04 · SaveSmart · Architecture

Flutter client · Riverpod · Repository pattern · REST over HTTPS

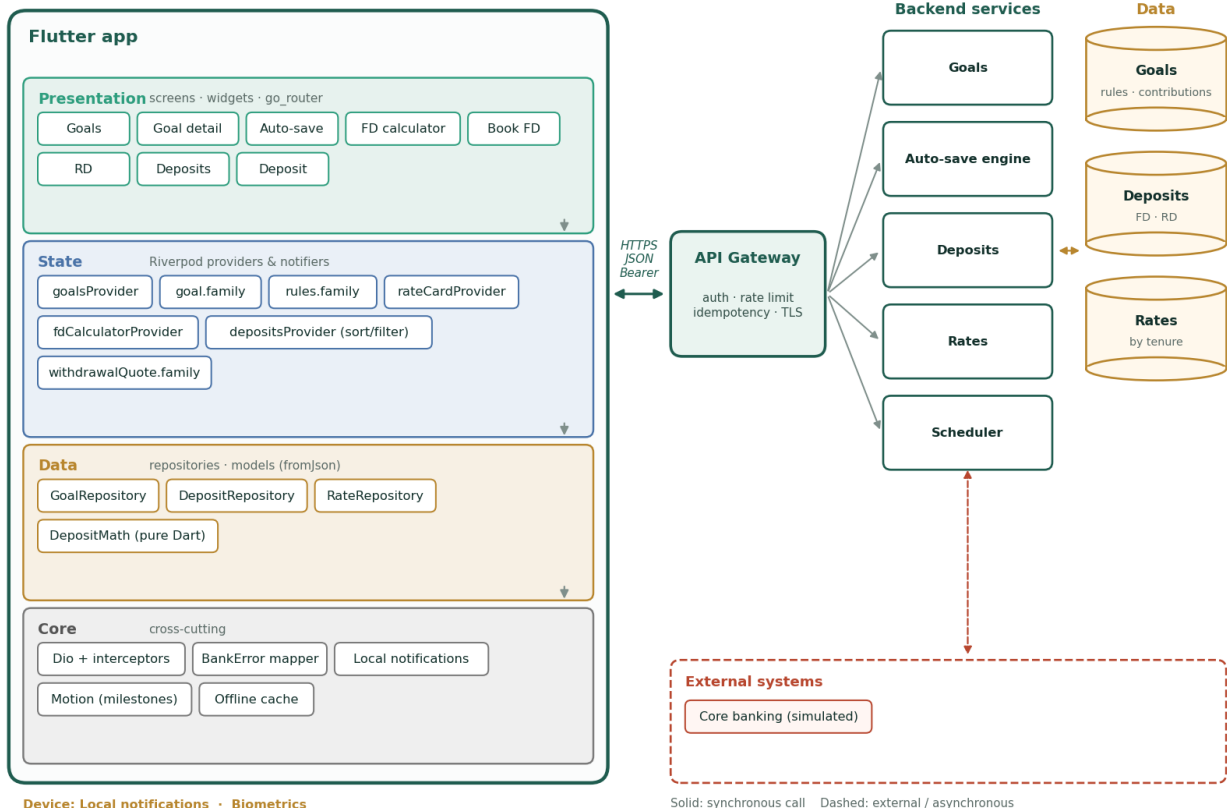


Figure P04 · SaveSmart architecture

- Presentation:** 8 screens (Goals, Goal detail, Auto-save, FD calculator, Book FD, RD, Deposits, Deposit), all routed through go_router with the session guard.
- State:** goalsProvider, goal.family, rules.family, rateCardProvider, fdCalculatorProvider, depositsProvider (sort/filter), withdrawalQuote.family. Screens *watch* these; buttons *read* their notifiers.
- Data:** GoalRepository, DepositRepository, RateRepository, DepositMath (pure Dart) — each repository owns one backend domain and throws only *BankError*.
- Core:** Dio + interceptors, BankError mapper, Local notifications, Motion (milestones), Offline cache.
- Backend (simulated):** Goals, Auto-save engine, Deposits, Rates, Scheduler, over Goals, Deposits, Rates.
- External systems (simulated):** Core banking (simulated). Treat them as unreliable: they can be slow, down, or return partial results.

10. Edge cases the solution must handle

- Rate card changes while the booking screen is open: re-quote before confirm
- Goal reached exactly: 100% animation once, then offer to create the next goal

3. Round-up rule on a ₹0.00 fractional transaction: no contribution
4. Withdrawal quote expired: fetch a new one, never use the old amount
5. RD debit date on the 29th–31st: disallowed
6. Offline with a stale portfolio: show "Last updated 2 h ago"

11. Stretch goals

- Shared family goals
- Tax-saver FD with 80C summary
- Charts of savings over time

12. Definition of done

Area	Done when...
Features	All Must features and baseline B1–B9 working against the mock API
The hard part	Financial maths that must be exactly right, plus motion that motivates without getting in the way.
Tests	Business rules unit-tested; widget tests for every Must screen; one integration test for the main journey; ≥ 70% coverage
Quality	Profiling numbers in the PR; skeletons; reduce motion; FLAG_SECURE where money or identity is shown
Delivery	CI green on protected main; signed, obfuscated release build; README with setup and this architecture

P05 · SpendWise

Expense analytics and monthly budgets

Theme	Personal finance
Level	Intermediate
Device features	Notifications · Share sheet
The hard part	Fast aggregation and charts over large data, and a recategorisation flow that feels instant yet stays consistent.

1. Problem statement

Context. Most customers have no idea where their money goes each month. Transactions are a long list of cryptic merchant strings.

Problem. Raw statements are unreadable, categories are missing, and there is no warning before a budget is exceeded.

Goal. Build SpendWise: auto-categorised transactions, monthly and category charts, budgets with alerts, merchant insights, and a way to fix wrong categories that the app remembers.

2. Users

Persona	Situation	What they need
Ananya, 26, spends mostly online	Wants to cut food delivery	See category totals and trends
Deepak, 38, household budget owner	Manages family expenses	Budgets per category with early warnings
Sara, 30, freelancer	Irregular income	Month-over-month comparison

3. Features

10 features: 5 Must · 3 Should · 2 Could — in addition to baseline requirements B1–B9.

ID	Feature	Description	Modules	Priority
F1	Categorised feed	Transactions with category icon, cleaned merchant name and amount, grouped by day.	M2, M4, M5	Must
F2	Recategorise and learn	Change a transaction's category; optionally apply to all from that merchant.	M3, M4	Must
F3	Monthly overview	Total spent vs last month, top categories donut chart, daily spend line chart.	M2, M5	Must
F4	Budgets	Monthly budget per category with progress bars coloured by risk.	M2, M3, M4	Must
F5	Budget alerts	Notification when a transaction pushes a category over 80% or 100%.	M4, M5	Should
F6	Merchant insights	Per-merchant totals, visit count and average spend.	M2, M4	Should
F7	Search and filters	Search merchants; filter by category, amount range and date range.	M2, M3	Must

ID	Feature	Description	Modules	Priority
F8	Month switcher	Swipe between months with cached data.	M3, M5	Should
F9	Export	Export a month as CSV via the share sheet.	M4	Could
F10	Insights cards	"You spent 32% more on food this month" style cards generated by the server.	M4	Could

4. Acceptance criteria

F1 · Categorised feed (Must)

- 1,000 transactions scroll at 60 fps
- Day headers stay sticky

F2 · Recategorise and learn (Must)

- "Apply to all" updates past and future matches
- Undo within 5 s

F3 · Monthly overview (Must)

- Charts render in under 300 ms for a month of data
- Tapping a slice filters the feed

F4 · Budgets (Must)

- At 80% the bar turns amber, at 100% red
- Budgets roll over monthly

F5 · Budget alerts (Should)

- At most one alert per category per threshold per month

F6 · Merchant insights (Should)

- Sorted by total by default

F7 · Search and filters (Must)

- Filters combine and survive navigating away and back

F8 · Month switcher (Should)

- Previously viewed months load instantly from cache

F9 · Export (Could)

- CSV has a header row and ISO dates

F10 · Insights cards (Could)

- Cards are dismissible

5. Screens and navigation

Screen	Route	Purpose
Overview	/overview	Charts and totals
Feed	/transactions	Categorised list
Transaction	/transactions/:id	Detail and recategorise
Budgets	/budgets	Progress bars
Budget edit	/budgets/:category	Set amount
Merchants	/merchants	Insights

Screen	Route	Purpose
Merchant	<code>/merchants/:id</code>	History
Filters	<code>sheet</code>	Filter sheet

6. Data model

Entity	Fields
Transaction	<code>id, merchantRaw, merchantName, category, amountPaise, at, mode</code>
Category	<code>id, name, icon, color</code>
Budget	<code>category, month, limitPaise, spentPaise</code>
MerchantRule	<code>merchantKey, category</code>
MonthSummary	<code>month, totalPaise, byCategory{}, byDay[]</code>

All money fields are integer paise; all timestamps are ISO-8601 UTC.

7. API contract

Method	Path	Purpose	Notable errors
GET	<code>/transactions?month=&category=&q=&cursor=</code>	Feed	—
PATCH	<code>/transactions/{id}</code>	Change category (+applyToMerchant)	422
GET	<code>/summary?month=</code>	Month summary	—
GET/PUT	<code>/budgets?month=</code>	Budgets	422
GET	<code>/merchants?month=</code>	Merchant insights	—
GET	<code>/insights</code>	Insight cards	—

8. Non-functional requirements

Quality	Requirement (in addition to the baseline)
Performance	Feed of 1,000 rows at 60 fps; month switch < 200 ms when cached; chart build off the UI thread when the month is large
Accuracy	Category totals equal the sum of their transactions to the paise
Privacy	Analytics computed per user; exports never leave the device except through the user's share action
Accessibility	Charts have a table alternative; colour is never the only signal
Usability	Recategorising takes at most 2 taps
Testability	Aggregation logic unit-tested; golden test for the donut; ≥ 70% coverage
Offline	Last three months viewable offline

9. Architecture

P05 · SpendWise · Architecture

Flutter client · Riverpod · Repository pattern · REST over HTTPS

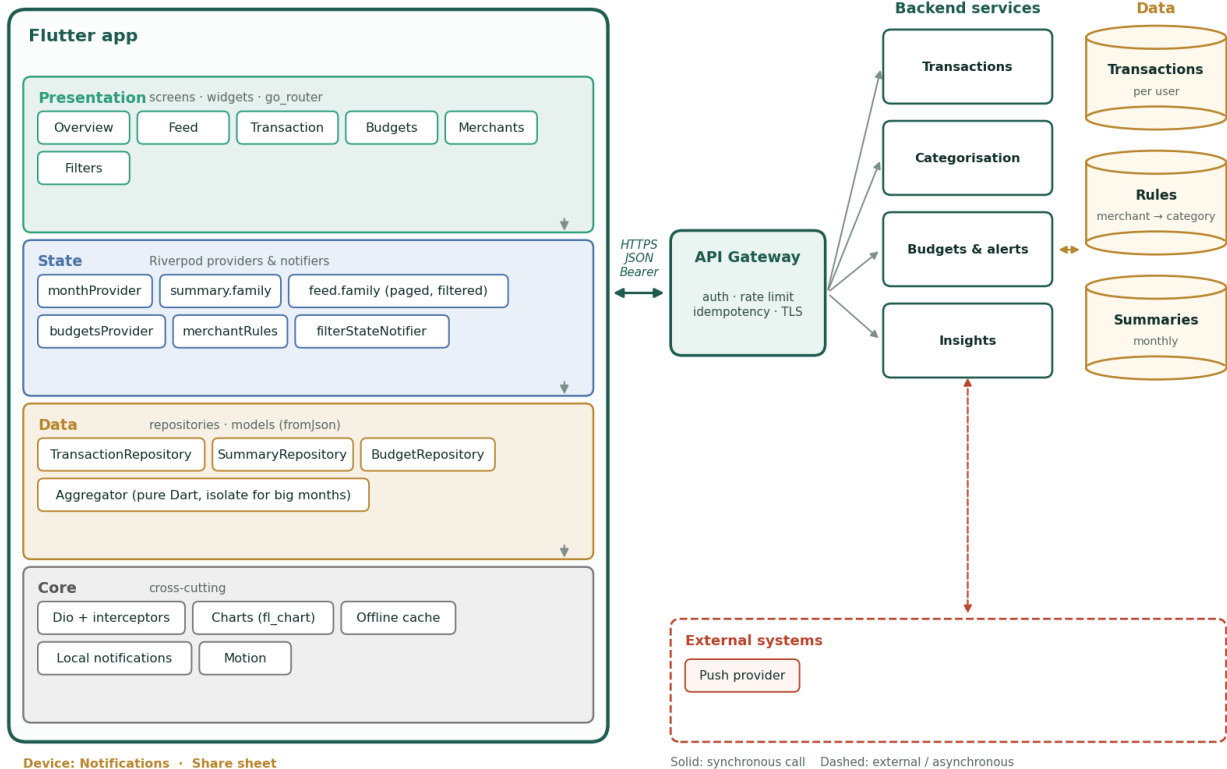


Figure P05 · SpendWise architecture

- **Presentation:** 6 screens (Overview, Feed, Transaction, Budgets, Merchants, Filters), all routed through go_router with the session guard.
- **State:** monthProvider, summary.family, feed.family (paged, filtered), budgetsProvider, merchantRules, filterStateNotifier. Screens watch these; buttons read their notifiers.
- **Data:** TransactionRepository, SummaryRepository, BudgetRepository, Aggregator (pure Dart, isolate for big months) — each repository owns one backend domain and throws only BankError.
- **Core:** Dio + interceptors, Charts (fl_chart), Offline cache, Local notifications, Motion.
- **Backend (simulated):** Transactions, Categorisation, Budgets & alerts, Insights, over Transactions, Rules, Summaries.
- **External systems (simulated):** Push provider. Treat them as unreliable: they can be slow, down, or return partial results.

10. Edge cases the solution must handle

1. A refund (positive amount) in a spend category reduces that category's total
2. A merchant string that changes slightly ("SWIGGY*1234"): normalise before matching rules
3. Budget set mid-month: progress counts the whole month
4. Zero transactions in a month: charts show an empty state, not a crash
5. Undo after "apply to all": restores every changed transaction
6. Very large month (5,000 rows): aggregation moves to an isolate

11. Stretch goals

- Receipt photo attached to a transaction
- Shared household budgets
- Predictive month-end spend

12. Definition of done

Area	Done when...
Features	All Must features and baseline B1–B9 working against the mock API
The hard part	Fast aggregation and charts over large data, and a recategorisation flow that feels instant yet stays consistent.
Tests	Business rules unit-tested; widget tests for every Must screen; one integration test for the main journey; $\geq 70\%$ coverage
Quality	Profiling numbers in the PR; skeletons; reduce motion; FLAG_SECURE where money or identity is shown
Delivery	CI green on protected main; signed, obfuscated release build; README with setup and this architecture

P06 · BillBuddy

Bill payments, recharges and autopay

Theme	Payments
Level	Intermediate
Device features	Notifications · Contacts (recharge) · Share sheet
The hard part	Server-driven dynamic forms, and many independent external systems that can each fail on their own.

1. Problem statement

Context. Electricity, water, broadband, mobile and credit card bills arrive on different dates through different channels. Late fees are common, and customers pay the same bills manually every month.

Problem. There is no single place to see what is due, reminders are unreliable, and setting up autopay means paperwork. Customers also cannot prove a payment when a biller claims it was not received.

Goal. Build BillBuddy: biller discovery, saved billers with fetched due amounts, one-tap payment, autopay with limits, reminders, and downloadable receipts.

2. Users

Persona	Situation	What they need
Lakshmi, 45, runs the household	Pays 7 bills a month	One list of what is due
Rohan, 29, forgetful	Paid 3 late fees last year	Autopay with a safety limit
Mr. Iyer, 67	Needs proof of payment	Receipts he can show the biller

3. Features

10 features: 7 Must · 3 Should · 0 Could — in addition to baseline requirements B1–B9.

ID	Feature	Description	Modules	Priority
F1	Biller directory	Browse by category (electricity, water, gas, broadband, mobile, DTH, credit card) and search by name and state.	M2, M4	Must
F2	Add a biller	Dynamic form per biller: consumer number, account ID, etc., validated by the biller's regex from the server.	M2, M3	Must
F3	Fetch bill	Fetch the current due amount, due date and bill period for a saved biller.	M4	Must
F4	Upcoming bills	Home list of saved billers sorted by due date with overdue/due-soon badges.	M2, M3	Must
F5	Pay bill	Pay exact or partial (if allowed) amount from a chosen account; idempotent; status success/pending/failed.	M4	Must
F6	Mobile recharge	Operator and circle auto-detected from number; plan browser with filters.	M2, M4, M5	Should
F7	Autopay	Enable autopay per biller with a maximum amount; payments above it require manual approval.	M3, M4	Should

ID	Feature	Description	Modules	Priority
F8	Reminders	Local reminders 3 days before due for billers without autopay.	M5	Should
F9	Receipts	Receipt with biller reference and bank reference; share as text or PDF.	M2, M4	Must
F10	Payment history	History per biller and overall with status filters.	M2, M4	Must

4. Acceptance criteria

F1 · **Biller directory** (Must)

- Search is debounced (300 ms)
- Categories load from the server, not hard-coded

F2 · **Add a biller** (Must)

- Fields and validations come from the biller definition
- A nickname is required

F3 · **Fetch bill** (Must)

- "No bill due" is a normal state, not an error
- Fetch failures offer retry

F4 · **Upcoming bills** (Must)

- Overdue first, then due within 3 days

F5 · **Pay bill** (Must)

- Partial payment is only offered for billers that allow it
- Retry never pays twice

F6 · **Mobile recharge** (Should)

- Plan list of 300 items scrolls smoothly
- Detected operator can be changed

F7 · **Autopay** (Should)

- Autopay can be paused
- Bills above the limit notify instead of paying

F8 · **Reminders** (Should)

- Reminders cancel when the bill is paid

F9 · **Receipts** (Must)

- Receipt available from history for 12 months

F10 · **Payment history** (Must)

- Pending payments update when refreshed

5. Screens and navigation

Screen	Route	Purpose
Home	/home	Upcoming bills
Categories	/billers	Browse
Search	/billers/search	Search
Add biller	/billers/:id/add	Dynamic form

Screen	Route	Purpose
Bill	/my-billers/:id	Fetches bill
Pay	/my-billers/:id/pay	Amount and account
Status	/payments/:id	Result
Recharge	/recharge	Number and plans
Autopay	/my-billers/:id/autopay	Settings
History	/history	Payments

6. Data model

Entity	Fields
Biller	id, name, category, state, fields[{key,label,regex}], allowsPartial
SavedBiller	id, billerId, nickname, params{}, autopay{enabled,maxPaise}
Bill	billerRef, amountPaise, dueDate, period, status
BillPayment	id, savedBillerId, amountPaise, status, bankRef, billerRef, at
Plan	id, operator, circle, pricePaise, validityDays, benefits

All money fields are integer paise; all timestamps are ISO-8601 UTC.

7. API contract

Method	Path	Purpose	Notable errors
GET	/biller-categories	Categories	—
GET	/billers?category=&q=&state=	Directory	—
POST	/my-billers	Save biller (server validates params)	422
GET	/my-billers/{id}/bill	Fetch current bill	404 NO_BILL, 502 BILLER_DOWN
POST	/bill-payments	Pay (Idempotency-Key)	422
PUT	/my-billers/{id}/autopay	Autopay settings	422
GET	/recharge/detect?number=	Operator and circle	—
GET	/recharge/plans?operator=&circle=	Plans	—

8. Non-functional requirements

Quality	Requirement (in addition to the baseline)
Reliability	Biller outages (502) degrade gracefully per biller, never for the whole app
Correctness	Amounts in paise; partial payment rules enforced client and server
Performance	Directory search results within 500 ms of typing stopping; plan list at 60 fps
Security	FLAG_SECURE on pay review; tokens in Keychain/Keystore
Usability	Pay a saved bill in 3 taps from home
Accessibility	Due badges have text, not just colour

Quality	Requirement (in addition to the baseline)
Testability	Dynamic form validation unit-tested per field type; $\geq 70\%$ coverage

9. Architecture

P06 · BillBuddy · Architecture

Flutter client · Riverpod · Repository pattern · REST over HTTPS

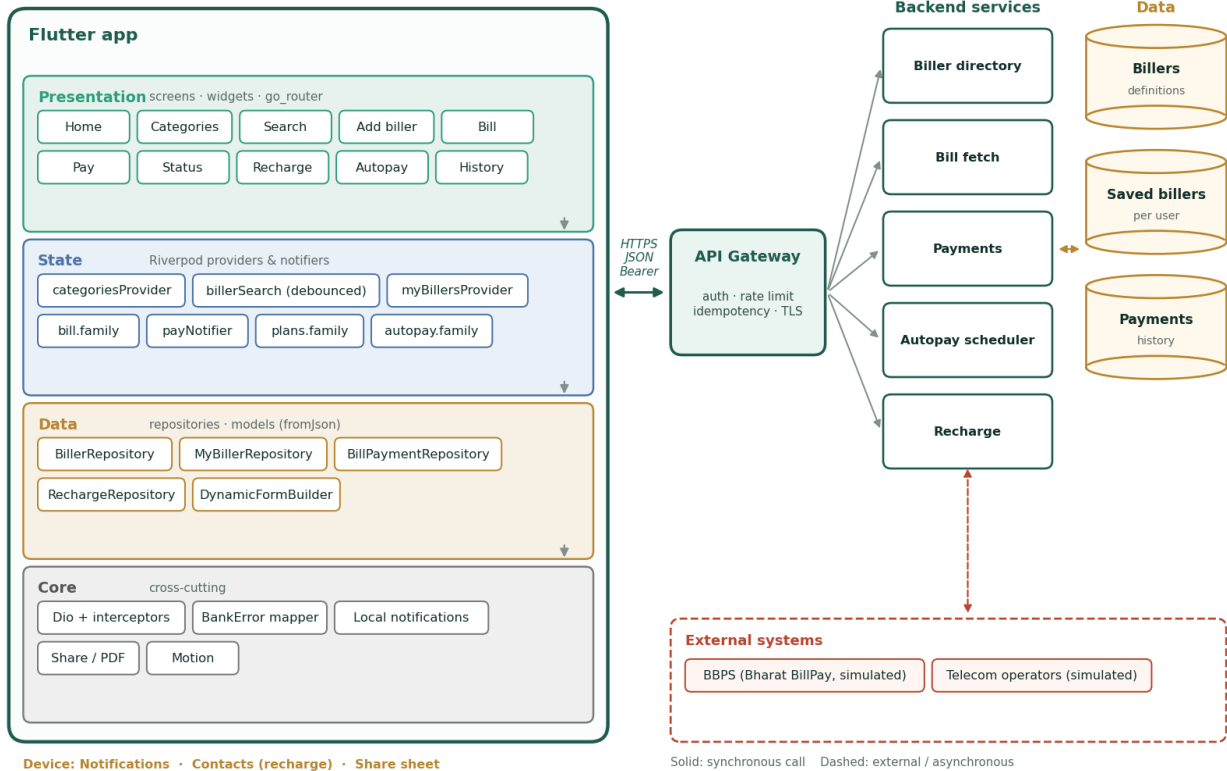


Figure P06 · BillBuddy architecture

- **Presentation:** 10 screens (Home, Categories, Search, Add biller, Bill, Pay, Status, Recharge, Autopay, History), all routed through go_router with the session guard.
- **State:** categoriesProvider, billerSearch (debounced), myBillersProvider, bill.family, payNotifier, plans.family, autopay.family. Screens **watch** these; buttons **read** their notifiers.
- **Data:** BillerRepository, MyBillerRepository, BillPaymentRepository, RechargeRepository, DynamicFormBuilder — each repository owns one backend domain and throws only **BankError**.
- **Core:** Dio + interceptors, BankError mapper, Local notifications, Share / PDF, Motion.
- **Backend (simulated):** Biller directory, Bill fetch, Payments, Autopay scheduler, Recharge, over Billers, Saved billers, Payments.
- **External systems (simulated):** BBPS (Bharat BillPay, simulated), Telecom operators (simulated). Treat them as unreliable: they can be slow, down, or return partial results.

10. Edge cases the solution must handle

1. Biller returns a due amount of ₹0: show "No bill due"
2. Consumer number valid by regex but unknown to the biller: server 422 shown on the field
3. Bill fetched yesterday, paid elsewhere today: re-fetch before pay
4. Autopay limit lower than the bill: notify, do not pay

5. Recharge number is a landline: operator detection fails with a manual choice
6. Pending payment older than 24 h: mark as "Under review" with support info

11. Stretch goals

- Pay multiple bills in one checkout
- Bill amount trend chart per biller
- Scan the bill's QR to add a biller

12. Definition of done

Area	Done when...
Features	All Must features and baseline B1–B9 working against the mock API
The hard part	Server-driven dynamic forms, and many independent external systems that can each fail on their own.
Tests	Business rules unit-tested; widget tests for every Must screen; one integration test for the main journey; $\geq 70\%$ coverage
Quality	Profiling numbers in the PR; skeletons; reduce motion; FLAG_SECURE where money or identity is shown
Delivery	CI green on protected main; signed, obfuscated release build; README with setup and this architecture

P07 · BranchQ

Branch appointments and live queue tokens

Theme	Branch services
Level	Beginner–Intermediate
Device features	GPS · Maps · Notifications
The hard part	Time-based, contended resources (slots and queues) with honest real-time updates that do not drain the battery.

1. Problem statement

Context. Branch visits are still needed for lockers, demand drafts, KYC updates and loan discussions. Customers wait an hour without knowing whether the branch is even open.

Problem. There is no way to find the right branch, see how busy it is, book a slot, or get a queue token before arriving.

Goal. Build BranchQ: find branches on a map, see services and live crowd levels, book an appointment slot, get a digital token with live position, and receive a "you are next" alert.

2. Users

Persona	Situation	What they need
Geeta, 58	Needs a locker visit	Book a slot and avoid the crowd
Ajay, 34, office worker	Lunch-break visits only	Know the wait before leaving the office
Branch manager	Runs 3 counters	Balanced load across the day

3. Features

9 features: 6 Must · 2 Should · 1 Could — in addition to baseline requirements B1–B9.

ID	Feature	Description	Modules	Priority
F1	Branch finder	List and map of nearby branches with distance, open/closed status and today's hours.	M2, M4	Must
F2	Branch detail	Address, hours, services, facilities and a live crowd indicator.	M2, M4	Must
F3	Service selection	Choose a service; each shows required documents and estimated time.	M2	Must
F4	Slot booking	Calendar of the next 7 days with 15-minute slots and remaining capacity.	M2, M3, M4	Must
F5	My appointments	Upcoming and past appointments; reschedule and cancel.	M2, M4	Must
F6	Walk-in token	Get a token for a service when at the branch (geofenced within 200 m).	M4, M5	Should
F7	Live queue	Token number, position, estimated wait and counter; refreshes every 20 s.	M3, M4	Must
F8	You-are-next alert	Notification when 2 tokens away and when called.	M5	Should

ID	Feature	Description	Modules	Priority
F9	Feedback	Rate the visit after completion.	M2, M4	Could

4. Acceptance criteria

F1 · Branch finder (Must)

- Location permission denied: search by city or PIN code
- Closed branches are clearly labelled

F2 · Branch detail (Must)

- Crowd level shown as text and colour

F3 · Service selection (Must)

- Required documents shown before booking

F4 · Slot booking (Must)

- Full slots are disabled
- Booking the last slot at the same moment as someone else returns a clear conflict (409)

F5 · My appointments (Must)

- Cancel up to 1 hour before

F6 · Walk-in token (Should)

- Outside 200 m: "You need to be at the branch"

F7 · Live queue (Must)

- Position updates without user action
- Stops polling when the screen is not visible

F8 · You-are-next alert (Should)

- Tapping opens the live queue

F9 · Feedback (Could)

- One rating per appointment

5. Screens and navigation

Screen	Route	Purpose
Finder	/branches	List and map
Branch	/branches/:id	Detail and crowd
Services	/branches/:id/services	Choose service
Slots	/branches/:id/book	Date and slot
Confirm	/bookings/confirm	Summary
My visits	/appointments	Upcoming and past
Token	/tokens/:id	Live queue
Feedback	/appointments/:id/feedback	Rating

6. Data model

Entity	Fields
Branch	id, name, address, lat, lng, hours[], services[], crowdLevel
Service	id, name, documents[], avgMinutes
Slot	start, capacity, remaining
Appointment	id, branchId, serviceId, slotStart, status
Token	id, number, position, etaMinutes, counter, status

All money fields are integer paise; all timestamps are ISO-8601 UTC.

7. API contract

Method	Path	Purpose	Notable errors
GET	/branches?lat=&lng=&q=	Nearby / search	—
GET	/branches/{id}	Detail	404
GET	/branches/{id}/slots?serviceId=&date=	Availability	—
POST	/appointments	Book (Idempotency-Key)	409 SLOT_FULL
PATCH	/appointments/{id}	Reschedule / cancel	409, 422
POST	/tokens	Walk-in token (lat/lng)	403 NOT_AT_BRANCH
GET	/tokens/{id}	Live position	—
POST	/appointments/{id}/feedback	Rating	409

8. Non-functional requirements

Quality	Requirement (in addition to the baseline)
Timeliness	Queue position at most 30 s stale while the token screen is open
Battery	No polling or location use in the background; polling pauses when the screen is hidden
Privacy	Location used only on the finder and token screens, with a clear reason
Performance	Map with 200 branch markers pans smoothly
Accessibility	Token number displayed very large; crowd level never colour-only
Testability	Slot conflict and polling lifecycle unit-tested
Offline	Upcoming appointments viewable offline

9. Architecture

P07 · BranchQ · Architecture

Flutter client · Riverpod · Repository pattern · REST over HTTPS

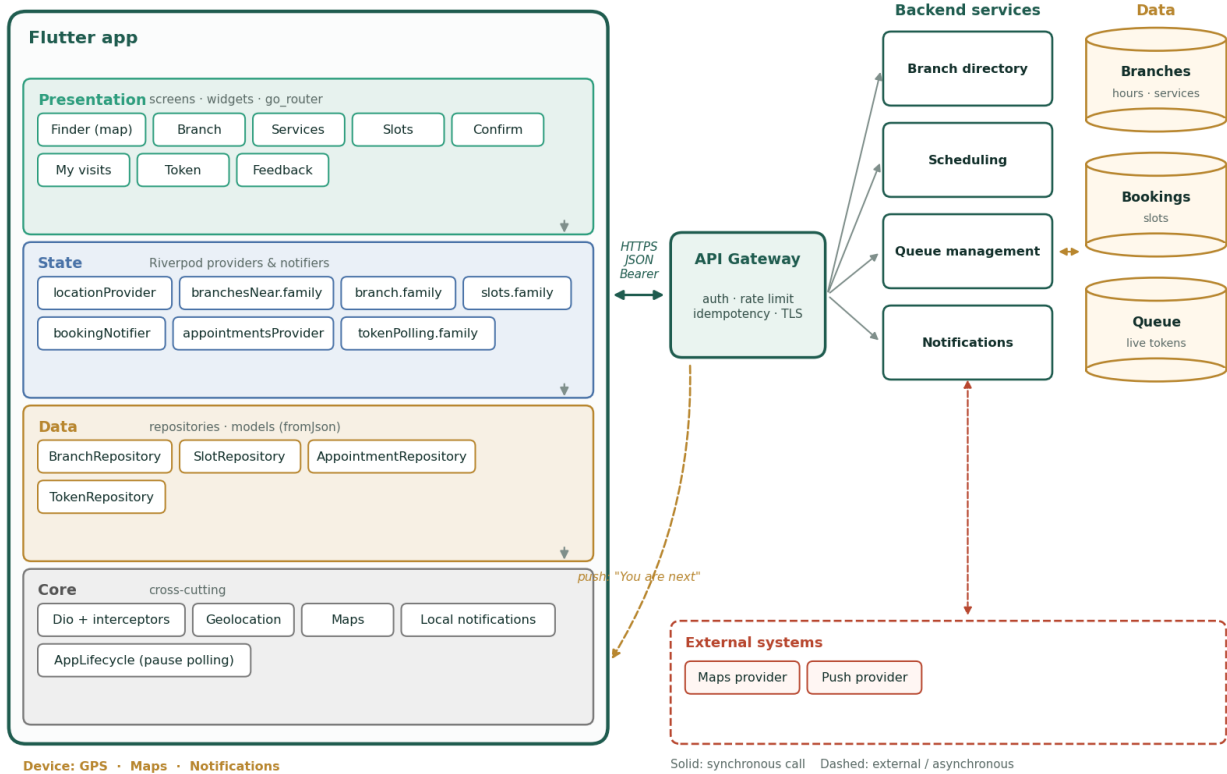


Figure P07 · BranchQ architecture

- **Presentation:** 8 screens (Finder (map), Branch, Services, Slots, Confirm, My visits, Token, Feedback), all routed through go_router with the session guard.
- **State:** locationProvider, branchesNear.family, branch.family, slots.family, bookingNotifier, appointmentsProvider, tokenPolling.family. Screens **watch** these; buttons **read** their notifiers.
- **Data:** BranchRepository, SlotRepository, AppointmentRepository, TokenRepository — each repository owns one backend domain and throws only **BankError**.
- **Core:** Dio + interceptors, Geolocation, Maps, Local notifications, AppLifecycle (pause polling).
- **Backend (simulated):** Branch directory, Scheduling, Queue management, Notifications, over Branches, Bookings, Queue.
- **External systems (simulated):** Maps provider, Push provider. Treat them as unreliable: they can be slow, down, or return partial results.
- **Push:** push: "You are next" — the notification opens the related screen through a deep link.

10. Edge cases the solution must handle

1. Two users book the last slot simultaneously: one gets 409 and a refreshed slot list
2. Branch closes early (holiday): appointments are cancelled with a notification
3. User arrives late and the token was skipped: show "Missed — rejoin queue"
4. Location is inaccurate indoors: allow a manual "I am at the branch" with a branch QR
5. Daylight/time-zone edge: all slot times from the server in IST, displayed as local
6. No branches within 50 km: widen search and show the nearest 5

11. Stretch goals

- Branch manager dashboard (web)
- Video appointment for loan discussions
- Accessibility services flag (wheelchair, assistance)

12. Definition of done

Area	Done when...
Features	All Must features and baseline B1–B9 working against the mock API
The hard part	Time-based, contended resources (slots and queues) with honest real-time updates that do not drain the battery.
Tests	Business rules unit-tested; widget tests for every Must screen; one integration test for the main journey; $\geq 70\%$ coverage
Quality	Profiling numbers in the PR; skeletons; reduce motion; FLAG_SECURE where money or identity is shown
Delivery	CI green on protected main; signed, obfuscated release build; README with setup and this architecture

P08 · OpenAccount

Digital savings account opening with KYC

Theme	Onboarding
Level	Advanced
Device features	Camera · SMS retriever (OTP) · Biometrics
The hard part	A long, regulated, interruptible state machine with sensitive data at every step.

1. Problem statement

Context. Opening a savings account still takes a branch visit and a stack of photocopies. Competing digital banks open accounts in 10 minutes.

Problem. Customers abandon the process at document collection, cannot tell what is pending, and data entered is re-typed by staff with errors.

Goal. Build OpenAccount: a guided onboarding flow with mobile OTP verification, PAN and Aadhaar-based KYC (simulated), selfie and signature capture, video-KYC scheduling, and a live status tracker ending in account activation.

2. Users

Persona	Situation	What they need
Tanvi, 21, first account	Joining college	Open an account from her phone
Harish, 39, switching banks	Busy, impatient	Finish in one sitting
Bank operations officer	Verifies applications	Clear, complete, legible data

3. Features

10 features: 8 Must · 2 Should · 0 Could — in addition to baseline requirements B1–B9.

ID	Feature	Description	Modules	Priority
F1	Mobile OTP	Enter mobile number, receive a 6-digit OTP (auto-read on Android), verify with retry limits.	M2, M4	Must
F2	PAN verification	Enter PAN; server returns the name on record for confirmation.	M2, M4	Must
F3	Aadhaar e-KYC (simulated)	Consent screen, Aadhaar number (masked as typed), OTP, then pre-filled name, DOB, address.	M2, M4	Must
F4	Personal details	Occupation, income band, nominee, communication address (same/different).	M2, M3	Must
F5	Selfie capture	Front camera with a face-oval guide; retake; liveness hint (blink) simulated.	M5	Must
F6	Signature capture	Draw a signature on a canvas; clear and redo; saved as PNG.	M2, M5	Should
F7	Account type and preferences	Choose account variant, debit card type, cheque book.	M2	Must

ID	Feature	Description	Modules	Priority
F8	Video-KYC slot	Book a video-KYC slot (date/time) or join now if an agent is available.	M4	Should
F9	Status tracker	Steps with states: Mobile ✓, PAN ✓, e-KYC ✓, Documents ✓, Video KYC  , Account active.	M2, M3, M4	Must
F10	Activation	Account number, IFSC, virtual debit card preview and set-MPIN.	M4, M5	Must

4. Acceptance criteria

F1 · Mobile OTP (Must)

- Resend enabled after 30 s
- 5 wrong attempts lock for 15 minutes

F2 · PAN verification (Must)

- PAN format validated before calling
- Name mismatch requires explicit confirmation

F3 · Aadhaar e-KYC (simulated) (Must)

- Aadhaar shown as XXXX XXXX 1234 after entry
- Pre-filled fields are read-only

F4 · Personal details (Must)

- Different address reveals a second address form

F5 · Selfie capture (Must)

- Image compressed below 500 KB
- Retake keeps the flow position

F6 · Signature capture (Should)

- Empty signature cannot be submitted

F7 · Account type and preferences (Must)

- Minimum balance shown per variant

F8 · Video-KYC slot (Should)

- Slots in the next 3 days only

F9 · Status tracker (Must)

- Resumable from any step after app restart

F10 · Activation (Must)

- MPIN never stored on the device

5. Screens and navigation

Screen	Route	Purpose
Welcome	/	Start or resume
Mobile	/onboard/mobile	Number
OTP	/onboard/otp	Verify
PAN	/onboard/pan	PAN
e-KYC	/onboard/kyc	Consent and OTP

Screen	Route	Purpose
Details	/onboard/details	Personal
Selfie	/onboard/selfie	Camera
Signature	/onboard/signature	Canvas
Account type	/onboard/variant	Choice
Video KYC	/onboard/vkyc	Slot
Status	/onboard/status	Tracker
Activated	/onboard/done	Welcome kit

6. Data model

Entity	Fields
OnboardingSession	id, step, mobile, panVerified, kycVerified, status
Applicant	name, dob, gender, address, occupation, incomeBand
Artifact	type (SELFIE/SIGNATURE), url, sizeBytes, status
VideoKycSlot	start, agentId
AccountResult	accountNumber, ifsc, cardPreview

All money fields are integer paise; all timestamps are ISO-8601 UTC.

7. API contract

Method	Path	Purpose	Notable errors
POST	/onboarding/otp	Send OTP	429 TOO_MANY
POST	/onboarding/otp/verify	Verify → session token	401, 423 LOCKED
POST	/onboarding/pan	Verify PAN	404, 422
POST	/onboarding/ekyc/otp /verify	Aadhaar e-KYC (simulated)	401, 422
PATCH	/onboarding/details	Save details	422
POST	/onboarding/artifacts	Upload selfie / signature	413, 415
GET	/onboarding/vkyc/slots	Slots	—
POST	/onboarding/submit	Submit (Idempotency-Key)	422 INCOMPLETE
GET	/onboarding/status	Tracker	—

8. Non-functional requirements

Quality	Requirement (in addition to the baseline)
Completion	Median completion under 10 minutes; resumable at every step
Security & privacy	Aadhaar never stored on device; masked in UI; images uploaded then deleted locally; FLAG_SECURE on all KYC screens
Compliance	Consent captured with timestamp and text version (server-side audit)
Reliability	OTP and uploads handle poor networks with retry and clear states

Quality	Requirement (in addition to the baseline)
Accessibility	Camera screens have text instructions and large capture buttons
Performance	Selfie compressed on-device in < 1 s
Testability	Step machine unit-tested; integration test through the full happy path with fakes

9. Architecture

P08 · OpenAccount · Architecture

Flutter client · Riverpod · Repository pattern · REST over HTTPS

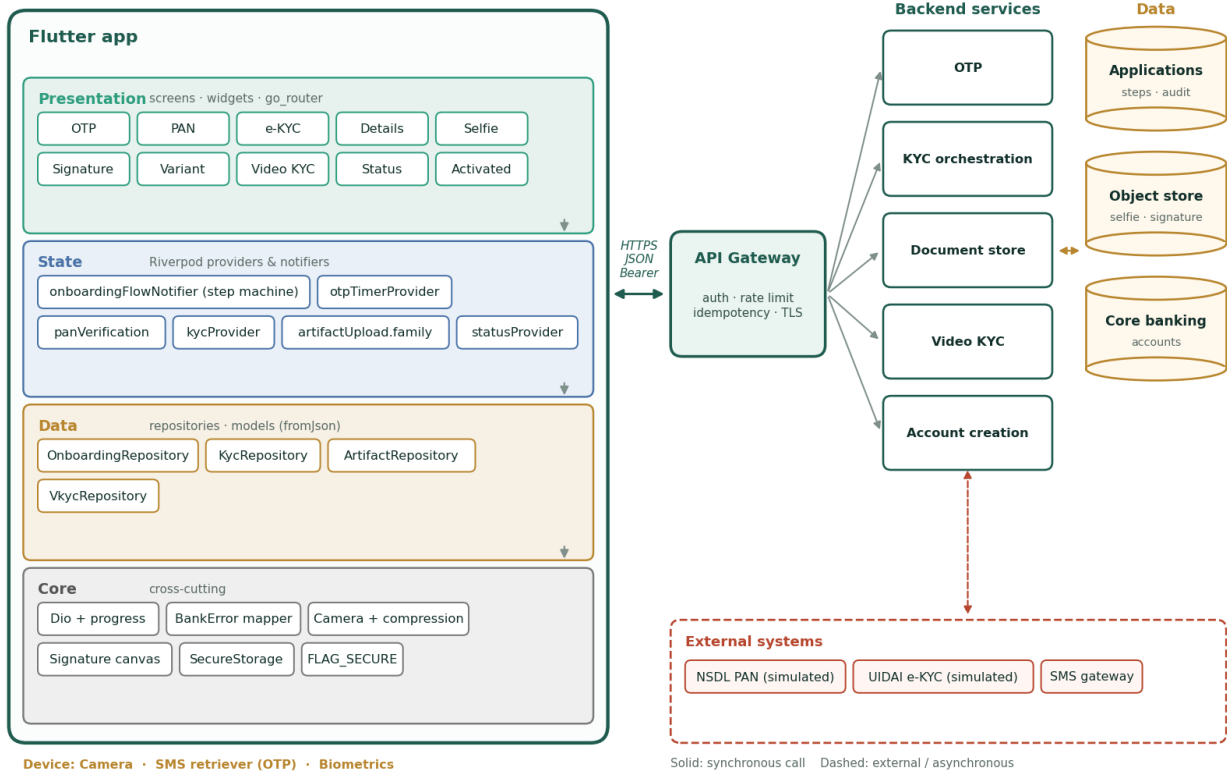


Figure P08 · OpenAccount architecture

- **Presentation:** 10 screens (OTP, PAN, e-KYC, Details, Selfie, Signature, Variant, Video KYC, Status, Activated), all routed through go_router with the session guard.
- **State:** onboardingFlowNotifier (step machine), otpTimerProvider, panVerification, kycProvider, artifactUpload.family, statusProvider. Screens `watch` these; buttons `read` their notifiers.
- **Data:** OnboardingRepository, KycRepository, ArtifactRepository, VkycRepository — each repository owns one backend domain and throws only `BankError`.
- **Core:** Dio + progress, BankError mapper, Camera + compression, Signature canvas, SecureStorage, FLAG_SECURE.
- **Backend (simulated):** OTP, KYC orchestration, Document store, Video KYC, Account creation, over Applications, Object store, Core banking.
- **External systems (simulated):** NSDL PAN (simulated), UIDAI e-KYC (simulated), SMS gateway. Treat them as unreliable: they can be slow, down, or return partial results.

10. Edge cases the solution must handle

1. OTP auto-read fills the wrong field: always allow manual entry

2. App killed during selfie upload: status shows the selfie as pending and allows retake
3. PAN name differs slightly from Aadhaar name ("Kumar S" vs "S Kumar"): flag for review, do not block
4. Video-KYC agent unavailable at slot time: automatic reschedule offer
5. User under 18 by Aadhaar DOB: stop with a minor-account message
6. Session expires mid-flow: re-verify mobile OTP and resume at the saved step

11. Stretch goals

- NFC reading of passport chip
- Instant virtual card usable for UPI immediately
- Referral codes

12. Definition of done

Area	Done when...
Features	All Must features and baseline B1–B9 working against the mock API
The hard part	A long, regulated, interruptible state machine with sensitive data at every step.
Tests	Business rules unit-tested; widget tests for every Must screen; one integration test for the main journey; $\geq 70\%$ coverage
Quality	Profiling numbers in the PR; skeletons; reduce motion; FLAG_SECURE where money or identity is shown
Delivery	CI green on protected main; signed, obfuscated release build; README with setup and this architecture

P09 · InvestLite

Mutual funds and SIP tracker

Theme	Wealth
Level	Advanced
Device features	Biometrics · Notifications
The hard part	Heavy charts and numeric correctness (XIRR, units, cut-offs), kept fast and honest.

1. Problem statement

Context. Retail investors hold mutual funds across several platforms and cannot see a single view of their portfolio or whether their SIPs are working.

Problem. Portfolio value, returns and allocation are scattered; starting or pausing a SIP is confusing; and NAV charts are slow and cluttered.

Goal. Build InvestLite: a unified portfolio with XIRR returns, fund discovery with filters, fund detail with fast NAV charts, lump-sum and SIP orders, SIP management, and a watchlist.

2. Users

Persona	Situation	What they need
Varun, 28, new investor	Starting a first SIP	Understand a fund before investing
Pooja, 41, experienced	8 funds, 4 SIPs	One view of allocation and returns
Mr. Menon, 60	Conservative	Clear risk labels

3. Features

10 features: 6 Must · 3 Should · 1 Could — in addition to baseline requirements B1–B9.

ID	Feature	Description	Modules	Priority
F1	Portfolio overview	Current value, invested amount, absolute and XIRR returns, allocation donut by category.	M2, M4	Must
F2	Holdings list	Per-fund units, NAV, value and return, sortable.	M2, M3, M5	Must
F3	Fund discovery	Browse by category (equity, debt, hybrid), risk level and rating; search by name.	M2, M4	Must
F4	Fund detail	NAV chart with 1M/6M/1Y/3Y/5Y ranges, returns table, risk-o-meter, expense ratio, holdings.	M2, M4, M5	Must
F5	Invest lump sum	Amount (min per fund), account, review, confirm (idempotent); order status.	M4	Must
F6	Start SIP	Amount, date (1–28), start month, optional step-up; mandate confirmation.	M2, M4	Must
F7	Manage SIPs	Pause (1–6 months), resume, modify amount, cancel with reason.	M3, M4	Should
F8	Watchlist	Add or remove funds; list with 1-day NAV change.	M3, M4	Should

ID	Feature	Description	Modules	Priority
F9	Redeem	Redeem by amount or all units; shows exit load and tax hint.	M4	Should
F10	Goal mapping	Tag holdings to goals and view progress.	M3	Could

4. Acceptance criteria

F1 · Portfolio overview (Must)

- XIRR matches a reference implementation within 0.01%
- Values refresh on pull

F2 · Holdings list (Must)

- Sort by value, return or name

F3 · Fund discovery (Must)

- Filters combine; results paged

F4 · Fund detail (Must)

- Range switch redraws in < 200 ms
- Chart downsamples data for long ranges

F5 · Invest lump sum (Must)

- Orders after the 3 pm cut-off show the next NAV date

F6 · Start SIP (Must)

- Minimum SIP per fund enforced

F7 · Manage SIPs (Should)

- Paused SIP shows its resume date

F8 · Watchlist (Should)

- Optimistic add/remove with rollback

F9 · Redeem (Should)

- Cannot exceed redeemable units

F10 · Goal mapping (Could)

- A holding can map to one goal

5. Screens and navigation

Screen	Route	Purpose
Portfolio	/portfolio	Overview
Holdings	/portfolio/holdings	List
Explore	/funds	Discovery
Fund	/funds/:id	Chart and details
Invest	/funds/:id/invest	Lump sum
SIP	/funds/:id/sip	Start SIP
My SIPs	/sips	Manage
Orders	/orders	Status
Watchlist	/watchlist	Saved funds
Redeem	/holdings/:id/redeem	Redeem

6. Data model

Entity	Fields
Fund	id, name, amc, category, risk, expenseRatio, minLumpPaise, minSipPaise
NavPoint	date, nav
Holding	fundId, units, investedPaise, currentPaise
Order	id, type (BUY/SELL), amountPaise, units, status, navDate
Sip	id, fundId, amountPaise, day, status, nextDate, stepUpPct

All money fields are integer paise; all timestamps are ISO-8601 UTC.

7. API contract

Method	Path	Purpose	Notable errors
GET	/portfolio	Summary + holdings	—
GET	/funds?category=&risk=&q=&cursor=	Discovery	—
GET	/funds/{id}	Detail	404
GET	/funds/{id}/nav?range=	NAV series (downsampled)	—
POST	/orders	Buy / redeem (Idempotency-Key)	422 BELOW_MIN, 422 INSUFFICIENT_UNITS
POST	/sips	Create SIP	422
PATCH	/sips/{id}	Pause / resume / modify	409
DELETE	/sips/{id}	Cancel	409
GET/PUT	/watchlist	Watchlist	—

8. Non-functional requirements

Quality	Requirement (in addition to the baseline)
Performance	NAV chart with 5 years of daily data renders in < 200 ms; downsampling to ≤ 500 points; parsing large series in an isolate
Correctness	XIRR and returns unit-tested against known cases; money in paise, units with 3–4 decimal precision (never double for money)
Transparency	Every order shows the NAV date and cut-off rule
Security	Orders confirmed with biometrics; FLAG_SECURE on order review
Accessibility	Charts expose a summary ("up 12.4% over 1 year"); risk labels in text
Testability	$\geq 75\%$ coverage; golden tests for the chart widget

9. Architecture

P09 · InvestLite · Architecture

Flutter client · Riverpod · Repository pattern · REST over HTTPS

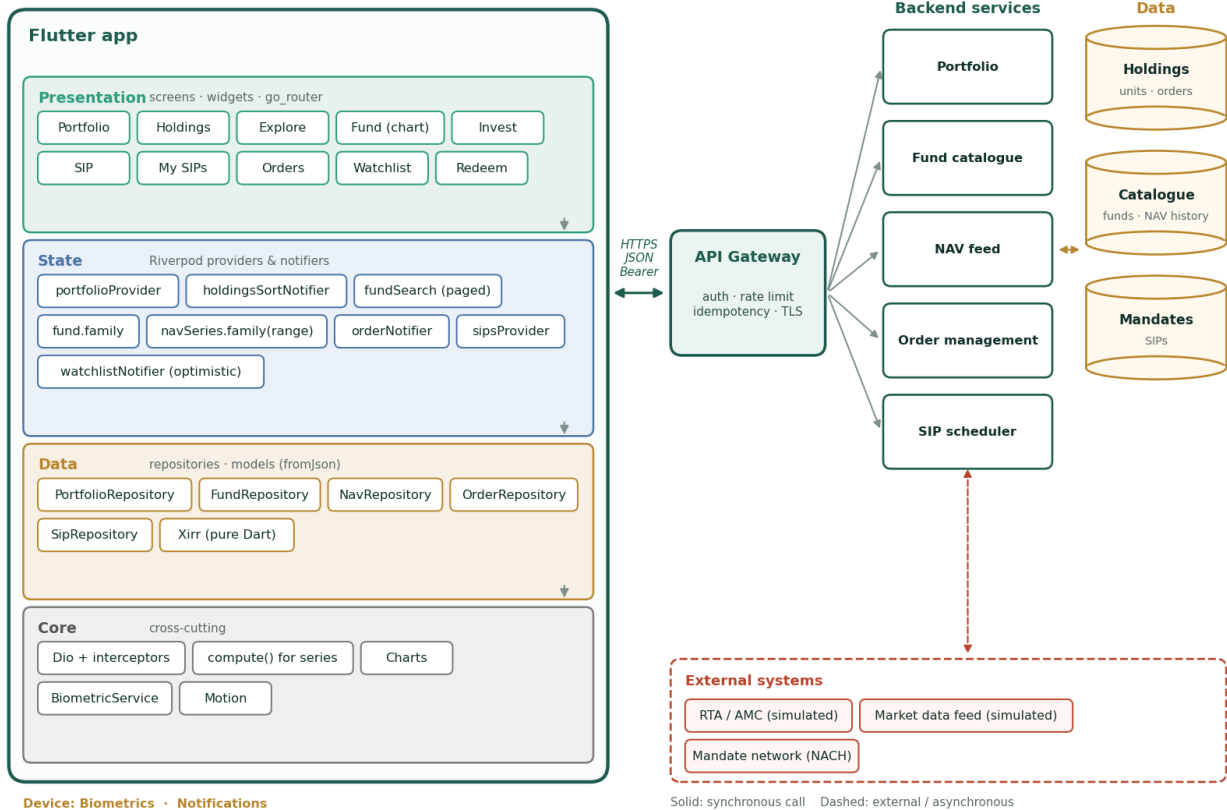


Figure P09 · InvestLite architecture

- **Presentation:** 10 screens (Portfolio, Holdings, Explore, Fund (chart), Invest, SIP, My SIPs, Orders, Watchlist, Redeem), all routed through go_router with the session guard.
- **State:** portfolioProvider, holdingsSortNotifier, fundSearch (paged), fund.family, navSeries.family(range), orderNotifier, sipsProvider, watchlistNotifier (optimistic). Screens **watch** these; buttons **read** their notifiers.
- **Data:** PortfolioRepository, FundRepository, NavRepository, OrderRepository, SipRepository, Xirr (pure Dart) — each repository owns one backend domain and throws only **BankError**.
- **Core:** Dio + interceptors, compute() for series, Charts, BiometricService, Motion.
- **Backend (simulated):** Portfolio, Fund catalogue, NAV feed, Order management, SIP scheduler, over Holdings, Catalogue, Mandates.
- **External systems (simulated):** RTA / AMC (simulated), Market data feed (simulated), Mandate network (NACH). Treat them as unreliable: they can be slow, down, or return partial results.

10. Edge cases the solution must handle

1. Order placed at 2:59 pm, confirmed at 3:01 pm: server decides the NAV date and the app shows it
2. Fund merged or closed: holdings show "Fund merged into ..."
3. SIP date falls on a holiday: next business day shown
4. Units with 4 decimals: display rounding never changes redeemable totals
5. Portfolio with zero holdings: onboarding empty state
6. NAV missing for a day: chart connects points without inventing values

11. Stretch goals

- Tax-harvesting suggestions
- Compare two funds side by side
- Portfolio rebalancing hints

12. Definition of done

Area	Done when...
Features	All Must features and baseline B1–B9 working against the mock API
The hard part	Heavy charts and numeric correctness (XIRR, units, cut-offs), kept fast and honest.
Tests	Business rules unit-tested; widget tests for every Must screen; one integration test for the main journey; $\geq 70\%$ coverage
Quality	Profiling numbers in the PR; skeletons; reduce motion; FLAG_SECURE where money or identity is shown
Delivery	CI green on protected main; signed, obfuscated release build; README with setup and this architecture

P10 · FraudShield

Real-time transaction alerts and dispute management

Theme	Risk & servicing
Level	Advanced
Device features	Push notifications · Biometrics · File picker
The hard part	Real-time streams, safety-critical actions that must be instant and idempotent, and a branching dispute flow with an auditable trail.

1. Problem statement

Context. Card and UPI fraud reports take days to resolve because customers call support, explain the transaction verbally, and the bank opens a case manually.

Problem. Customers learn about suspicious transactions late, cannot act immediately, and have no visibility into dispute progress, which drives repeated calls and complaints to the ombudsman.

Goal. Build FraudShield: real-time alerts for suspicious transactions with "Yes, it was me / No, block it" actions, instant card and UPI blocking, a guided dispute flow with evidence upload, and a case tracker with SLA timers.

2. Users

Persona	Situation	What they need
Nisha, 33	Gets an alert for a transaction she did not make	Stop it in one tap
Rahul, 26	Online purchase never delivered	Raise a dispute with proof
Support agent	Handles disputes	Complete, structured cases

3. Features

9 features: 5 Must · 3 Should · 1 Could — in addition to baseline requirements B1–B9.

ID	Feature	Description	Modules	Priority
F1	Alert inbox	Real-time alerts (push + in-app list) for transactions flagged as unusual, with merchant, amount, location and time.	M3, M4, M5	Must
F2	Confirm or deny	"Yes, it was me" clears the alert; "No, it wasn't me" blocks the instrument and starts a dispute.	M2, M3, M4	Must
F3	Instant block	Block a card, UPI or net banking access with a reason; show which instruments are blocked.	M4, M5	Must
F4	Raise a dispute	Guided flow: pick transaction → reason (unauthorised, not received, duplicate, wrong amount) → questions per reason → evidence → review.	M2, M3	Must
F5	Evidence upload	Attach screenshots, invoices, chats (images/PDF) with progress and retry.	M4, M5	Should
F6	Case tracker	Case number, status timeline, SLA countdown (e.g., 10 days for provisional credit), and messages from the bank.	M2, M3, M4	Must
F7	Provisional credit	When granted, show the credit and conditions on the case.	M4	Should

ID	Feature	Description	Modules	Priority
F8	Secure messages	Two-way messages on a case (text + attachments).	M3, M4	Could
F9	Security centre	Login history, active devices, sign out other devices.	M4, M5	Should

4. Acceptance criteria

F1 · Alert inbox (Must)

- New alerts appear without refresh while the app is open
- Unread count on the tab

F2 · Confirm or deny (Must)

- Deny requires biometric confirmation
- Deny completes in under 2 s end to end

F3 · Instant block (Must)

- Block is idempotent
- Blocked state visible on the home screen

F4 · Raise a dispute (Must)

- Questions change by reason
- Evidence optional for "unauthorised", required for "not received"

F5 · Evidence upload (Should)

- Max 5 files, 5 MB each

F6 · Case tracker (Must)

- SLA timer shows days remaining and turns amber in the last 2 days

F7 · Provisional credit (Should)

- Credit amount matches the disputed amount or shows the difference

F8 · Secure messages (Could)

- Messages paginate; newest at bottom

F9 · Security centre (Should)

- Current device cannot be removed from itself

5. Screens and navigation

Screen	Route	Purpose
Home	<code>/home</code>	Alerts summary and blocked instruments
Alerts	<code>/alerts</code>	Inbox
Alert	<code>/alerts/:id</code>	Confirm / deny
Block	<code>/block</code>	Instruments
Dispute	<code>/disputes/new/:txnId</code>	Guided flow
Evidence	<code>/disputes/new/evidence</code>	Upload
Cases	<code>/cases</code>	List
Case	<code>/cases/:id</code>	Timeline, SLA, messages
Security	<code>/security</code>	Devices and logins

6. Data model

Entity	Fields
Alert	id, txnId, merchant, amountPaise, location, at, riskReason, status
Instrument	id, type (CARD/UPI/NETBANKING), masked, blocked
Dispute	id, txnId, reason, answers{}, evidence[], status
Case	id, disputeId, status, timeline[], slaDueAt, provisionalCreditPaise
Device	id, name, lastSeen, current

All money fields are integer paise; all timestamps are ISO-8601 UTC.

7. API contract

Method	Path	Purpose	Notable errors
GET	/alerts?status=	Inbox	—
POST	/alerts/{id}/confirm	It was me	409
POST	/alerts/{id}/deny	Not me → block + dispute (Idempotency-Key)	409
POST	/instruments/{id}/block	Block	409
POST	/disputes	Create	422 by reason
POST	/disputes/{id}/evidence	Upload	413
GET	/cases/{id}	Case with timeline	404
GET/POST	/cases/{id}/messages	Messages	—
GET	/devices	Devices	—
DELETE	/devices/{id}	Sign out a device	409 CURRENT

8. Non-functional requirements

Quality	Requirement (in addition to the baseline)
Latency	Alert visible within 5 s of the server flagging it (push or in-app stream); deny-and-block under 2 s
Security	Deny, block and device removal require biometrics; FLAG_SECURE on all screens; no transaction details in notification previews when the device is locked
Reliability	Block and deny are idempotent and safe to retry on flaky networks
Auditability	Every action carries a client timestamp and device id; server keeps an immutable log
Accessibility	Alert actions are large, clearly labelled buttons; colour plus icon plus text for risk
Testability	Dispute question logic unit-tested per reason; integration test for deny → block → case created

9. Architecture

P10 · FraudShield · Architecture

Flutter client · Riverpod · Repository pattern · REST over HTTPS

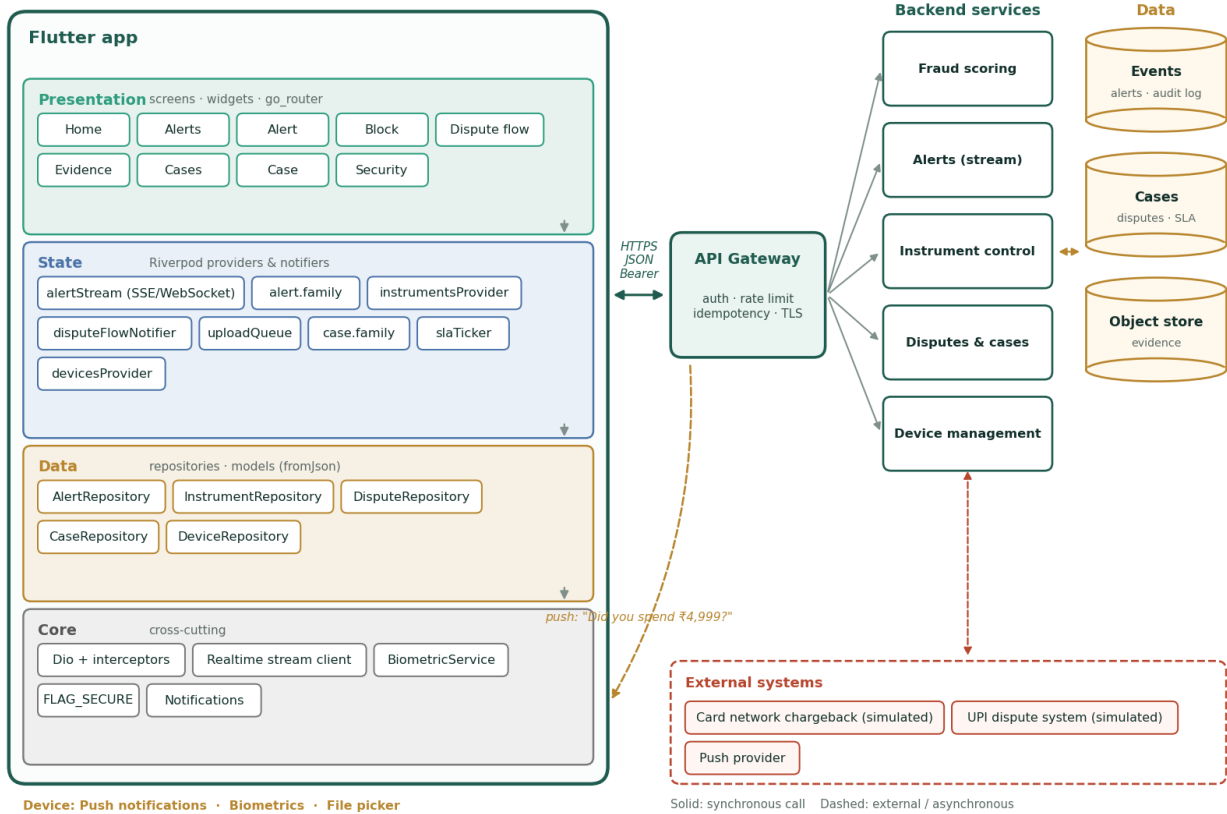


Figure P10 · FraudShield architecture

- **Presentation:** 9 screens (Home, Alerts, Alert, Block, Dispute flow, Evidence, Cases, Case, Security), all routed through go_router with the session guard.
- **State:** alertStream (SSE/WebSocket), alert.family, instrumentsProvider, disputeFlowNotifier, uploadQueue, case.family, slaTicker, devicesProvider. Screens **watch** these; buttons **read** their notifiers.
- **Data:** AlertRepository, InstrumentRepository, DisputeRepository, CaseRepository, DeviceRepository — each repository owns one backend domain and throws only **BankError**.
- **Core:** Dio + interceptors, Realtime stream client, BiometricService, FLAG_SECURE, Notifications.
- **Backend (simulated):** Fraud scoring, Alerts (stream), Instrument control, Disputes & cases, Device management, over Events, Cases, Object store.
- **External systems (simulated):** Card network chargeback (simulated), UPI dispute system (simulated), Push provider. Treat them as unreliable: they can be slow, down, or return partial results.
- **Push:** push: "Did you spend ₹4,999?" — the notification opens the related screen through a deep link.

10. Edge cases the solution must handle

1. Deny tapped twice on a slow network: one block, one case
2. Alert arrives for an already-blocked card: show it as "Blocked — no action needed"
3. The stream disconnects: reconnect with backoff and fetch missed alerts since the last id
4. Dispute for a transaction older than 90 days: explain the window and offer support contact
5. Evidence upload fails at 90%: retry that file only
6. SLA breached: case highlights it and shows the escalation path

11. Stretch goals

- On-device risk hints (unusual location vs last login)
- Chatbot triage before the dispute form
- Temporary card unblock for a single merchant

12. Definition of done

Area	Done when...
Features	All Must features and baseline B1–B9 working against the mock API
The hard part	Real-time streams, safety-critical actions that must be instant and idempotent, and a branching dispute flow with an auditable trail.
Tests	Business rules unit-tested; widget tests for every Must screen; one integration test for the main journey; $\geq 70\%$ coverage
Quality	Profiling numbers in the PR; skeletons; reduce motion; FLAG_SECURE where money or identity is shown
Delivery	CI green on protected main; signed, obfuscated release build; README with setup and this architecture

Appendix · Writing acceptance tests from these criteria

Each acceptance criterion is written to become a test. The pattern learners used in Module 6 applies directly:

```
// Criterion (PayLite F5): "A retry after a 30-second timeout returns the original receipt"
test('retry with the same idempotency key returns the original receipt', () async {
  // Arrange: fake API that times out on the first call, succeeds on the second
  // Act:      pay twice with one key
  // Assert:   same reference id, balance debited once
});
```

If the criterion is about...	Write a...
A pure rule (maths, validation)	Unit test, no Flutter
A repository or notifier behaviour	Unit test with a fake adapter and ProviderContainer
What the user sees on one screen	Widget test with testApp(...)
A journey across screens	Widget flow test, then one integration test on a device